Sheet: Sheet1

- Paper File Name : A Systematic Literature Review of A* Pathfinding

- Paper Name: nan

- Citation: Foead, D., Ghifari, A., Kusuma, M. B., Hanafiah, N., & Gunawan, E. (2021). A Systematic Literature Review of A* Pathfinding. Procedia Computer Science, 179, 507–514. 5th International Conference on Computer Science and Computational Intelligence 2020.

- Problem/Gap: • The large volume of research on A* makes it difficult for new researchers to obtain a clear, up-to-date overview of its current state, modifications, and effectiveness. • The classic A* algorithm struggles with modern pathfinding demands, especially on large maps where performance degrades and overhead increases.

- Algorithm/Method: • Systematic Literature Review (SLR) of 40 quantitative research papers on A* from the past 5 years.

- Categorizes and analyzes common A* variants, including:
  - Bidirectional A*
  - Iterative Deepening A* (IDA*)
  - Hierarchical Path-Finding A* (HPA*)
  - Dynamically Weighted A* (dwA*)
  - Local Repair A* (LRA*)

- Heuristic/Obstacle Handling: • Discusses common distance-based heuristics: Manhattan, Euclidean, Diagonal.

- Notes that modifying heuristic weighting or structure is a primary method for improving A*.
- Mentions direction-based heuristics that significantly reduce computation time (1–2 ms vs. 6–7 ms on 150×150 grids).
- Reviews swarm-intelligence approaches (bee-inspired) for multi-agent collision and path handling.

- Key Results : • A* variants can be over 40% more efficient than other algorithms in specific cases.

- Bidirectional A* performs poorly on larger grids (16×16) but is faster on smaller grids (8×8).
- HPA* is extremely fast but may not always return optimal paths.
- Depth Direction A* (DepthD A*) shows a 50% speed improvement and 28.6% reduction in node expansion.
- Euclidean distance heuristics showed over 100% efficiency improvement in one

study.

- Limitations/Open Issues: • High memory requirement for maintaining open and closed lists.
• Performance strongly depends on heuristic accuracy; poor heuristics lead to suboptimal outcomes.
• Bidirectional A* is less effective on large-scale graphs compared with uninformed methods like BFS.
• Adapting A* for multi-agent pathfinding (MAPF) remains challenging due to collision-avoidance complexity.

- Relevance to my Research: • Confirms that classic A* is inadequate for modern applications and requires enhancements.
• Identifies Bidirectional A* performance issues on large grids, offering a clear gap for new research.
• Supports methods involving direction-based heuristics and obstacle-aware penalty adjustments.
• Highlights the speed–optimality trade-off relevant to real-time navigation in urban environments.

- Paper File Name : 1408-012

- Paper Name: nan

- Citation: · Ziliaskopoulos, A. K., & Mahmassani, H. S. (1993). Time-dependent, shortest-path algorithm for real-time intelligent vehicle highway system applications. Transportation Research Record, 1408.

- Problem/Gap: • Need for an efficient algorithm to compute time-dependent shortest paths in large, real-time traffic networks (IVHS).
• Existing algorithms were too slow for real-time use or unable to handle networks where travel times do not follow the First-In-First-Out (FIFO) property.

- Algorithm/Method: • A label-correcting algorithm computing shortest paths from all nodes to a single destination over a discretized time horizon.
• Based on Bellman’s principle of optimality, operating backward from the destination.
• Does not require the FIFO property, unlike Dreyfus’s extension of Dijkstra’s algorithm.

- Uses a double-ended queue (deque) to efficiently manage nodes to be scanned.

- Heuristic/Obstacle Handling: • Handles dynamic congestion through time-dependent arc costs (travel times).
- Does not use a heuristic function; exploration is label-correcting rather than goal-directed.

- Key Results : • On a real-world Austin network (625 nodes, 1,724 arcs, 503 time intervals), average computation time was 107.41 ms on a CRAY Y-MP/8 supercomputer.
- Very high efficiency: number of scanned nodes was only 1.18 × the number of nodes in the network.
- Computation time scales nearly linearly with the number of nodes and time steps.

- Limitations/Open Issues: • Unidirectional (all-to-one) rather than point-to-point or bidirectional, limiting direct use for individual vehicle navigation.
- Efficiency depends on network characteristics, such as limited optimal paths over time.
- Memory demands may be high for very large networks with many time intervals.

- Relevance to my Research: • Contrast: Uses a unidirectional, label-correcting approach, unlike the heuristic-guided Bidirectional A* focus of my work.
- Inspiration: Supports modeling dynamic navigation conditions (e.g., traffic) as time-dependent arc costs; aligns with obstacle-aware penalties.
- Gap: Since it is all-to-one, it leaves an opening for faster point-to-point methods like Bidirectional A*, positioning my research as a more direct solution for individual navigation queries.

- Paper File Name : 4_AoT_72(4)_DW_75-88.pdf

- Paper Name: nan

- Citation: · Wang, D., & Jing, Y. (2024). Obstacle avoidance for ship navigation safety combining heuristic search algorithm and improved ACO algorithm. Archives of Transport, 72(4), 75–88.

- Problem/Gap: • Traditional maritime obstacle-avoidance methods rely heavily on human experience.
- Standard algorithms have limitations: ACO easily falls into local optima, and A*

struggles with large search spaces.

- Algorithm/Method: • Proposes a hybrid algorithm (HA) combining an Improved Ant Colony Optimization Algorithm (IACOA) with the A* algorithm.
- Improved ACO (IACOA):
  - Adds a stabilization factor to reduce the chance of selecting paths near obstacles, preventing local optima.
  - Includes a cornering factor for smoother paths.
  - Uses a reward–punishment mechanism for pheromone updates to accelerate convergence.
- Hybridization (A + IACOA):*
  - A* generates an initial high-quality path.
  - Pheromone trails in IACOA are initialized along this A* path to guide and speed up optimization.

- Heuristic/Obstacle Handling: • Heuristic: A* uses Manhattan distance.
- Obstacle Handling:
  - Environment modeled as a grid map.
  - IACOA uses a stabilization factor to penalize routes near obstacles by lowering their selection probability.
  - The A* initial path supplies a globally valid, obstacle-free guide for IACOA refinement.

- Key Results : • Hybrid algorithm (HA) outperforms PSO and LOS in environments of varying complexity.
- Complex environments:
  - Path length reduced by 15.7 (vs. PSO) and 12.4 (vs. LOS).
  - Iterations reduced by 49 (vs. PSO) and 22.2 (vs. LOS).
- Simple environments:
  - Path length reduced by 5.5 (vs. PSO) and 3.8 (vs. LOS).
  - Iterations reduced by 30.7 (vs. PSO) and 13.2 (vs. LOS).

- Limitations/Open Issues: • Validated only in simulation.
- Requires real-world validation through actual ship-navigation tests.

- Relevance to my Research: • Contrast: Uses maritime navigation and a unidirectional A* + ACO hybrid, not Bidirectional A*.
- Obstacle-handling insight: The stabilization factor is a direct form of obstacle-aware penalty; its density-based calculation could be adapted to a Bidirectional A* heuristic.
- Methodological insight: Hybridization (using A* to generate an initial solution that

accelerates another algorithm) offers a concept adaptable for new variants.
- Experimental insight: Their tiered testing (simple, moderate, complex environments) provides a solid framework for designing experiments in my own work.

- Paper File Name : 2012-56

- Paper Name: nan

- Citation: • Oroko, J. A., & Nyakoe, G. N. (2012). Obstacle Avoidance and Path Planning Schemes for Autonomous Navigation of a Mobile Robot: A Review. Proceedings of the 2012 Mechanical Engineering Conference on Sustainable Research and Innovation (Vol. 4).

- Problem/Gap: • Reviews three widely used local, sensor-based obstacle avoidance techniques for robots in unknown or partially known environments.
- Does not propose a new algorithm—focuses on comparison and analysis.

- Algorithm/Method: • Bug Algorithms:
o Robot moves straight toward the goal; when an obstacle is encountered, it contours around it until it can resume its original direction.
- Artificial Potential Field (APF):
o Robot is treated as a particle influenced by attractive (goal) and repulsive (obstacles) forces.
o Movement follows the negative gradient of the potential field.
- Vector Field Histogram (VFH):
o Uses a 2D occupancy grid converted into a 1D polar histogram representing obstacle density.
o Selects a steering direction through low-density "valleys" closest to the goal direction.

- Heuristic/Obstacle Handling: • Bug: No heuristic; purely reactive contour-following.
- APF:
o Heuristic is the attractive force toward the goal.
o Repulsive forces manage obstacle avoidance.
o Susceptible to local minima (e.g., U-shaped traps).
- VFH:
o Converts local obstacle data into a density histogram.
o Selects the lowest-density feasible direction.
o Avoids APF-style local minima but may deviate significantly from the goal.

- Key Results : • As a review paper, it presents no new experimental or quantitative results.

- Bug: Guarantees reaching the goal if possible but produces inefficient, lengthy paths.
- APF: Efficient in open spaces but fails in complex environments due to local minima.
- VFH: More robust in cluttered environments and avoids local minima; may still yield suboptimal or goal-divergent paths.

- Limitations/Open Issues: • All three techniques are local planners and therefore do not guarantee globally optimal paths.
- APF: Local minima traps.
- VFH: May lead robot far from the goal if clear paths point away from it.
- Suggests hybrid approaches—e.g., combining methods or integrating vision—for better robustness.

- Relevance to my Research: • Provides foundational context on local, reactive obstacle avoidance. Although my Bidirectional A* is a global path planner, it ultimately must pair with a local planner for real-time navigation.
- The idea of obstacle-aware penalties parallels concepts in APF (repulsion) and VFH (density-based steering) and can inform how to incorporate obstacle information into A*'s cost function.
- My work differs by proactively shaping the global path using penalties rather than reacting to obstacles during execution.

- Paper File Name : agarwal2006

- Paper Name: nan

- Citation: • Agarwal, A., Colak, S., & Eryarsoy, E. (2006). Improvement heuristic for the flow-shop scheduling problem: An adaptive-learning approach. European Journal of Operational Research, 169(3), 801–815.

- Problem/Gap: • Focuses on the NP-hard flow-shop scheduling problem (n/m/Cmax), where exact methods become impractical for large instances.
- Aims to develop an improvement heuristic that can efficiently produce optimal or near-optimal makespan solutions.

- Algorithm/Method: • Adaptive Learning Approach (ALA):
o Inspired by neural-network learning processes.
o Starts with a good initial solution generated by heuristics such as Palmer, CDS, or NEH.
o Perturbs processing times using operation-specific weights (w_{ij}).
o Weights are iteratively updated through a reinforcement-and-backtracking learning

strategy:
- Improvement → reinforce weight changes.
- No improvement for many iterations → reverse (backtrack) the weight adjustments.
o Perturbation introduces controlled randomness to escape local optima, enabling a non-deterministic local search.

- Heuristic/Obstacle Handling: • No spatial obstacles or pathfinding heuristics.
• Perturbation combined with adaptive learning serves as a meta-heuristic to explore the solution space more effectively.

- Key Results : • Evaluated on multiple benchmark sets (Taillard, Carlier, Heller, Reeves).
• NEH-ALA matched best-known upper bounds on many instances (e.g., 24/30 of Taillard's 5-machine problems).
• Discovered one new upper bound on Taillard's dataset.
• Average gaps from best known:
o Taillard: 1.74%
o Carlier: 0%
o Reeves: 1.52%
• Computation time scales heavily with problem size:
o 20×5 instance: ~34 seconds
o 100×5 instance: ~3736 seconds

- Limitations/Open Issues: • Purely a scheduling problem—results cannot be applied directly to pathfinding.
• Learning parameters (e.g., reinforcement factor, learning rate, TINI) require empirical tuning and may not generalize well.
• Suggests extending the method to more complex variations of flow-shop problems.

- Relevance to my Research: • Methodological Inspiration:
o The idea of iteratively adjusting weights based on performance is directly applicable to tuning obstacle-aware penalties in Bidirectional A*.
• Meta-Heuristic for Penalty Tuning:
o A similar feedback-driven mechanism could dynamically increase or decrease penalties based on path quality across simulations.
• Contrast:
o The domains differ (scheduling vs. spatial pathfinding).
o However, the adaptive learning mechanism offers a transferable conceptual tool for improving heuristic-driven search algorithms.

- Paper File Name : 863955.863962.pdf

- Paper Name: nan

- Citation: • Basu, A., Lin, A., & Ramanathan, S. (2003). Routing Using Potentials: A Dynamic Traffic-Aware Routing Algorithm. SIGCOMM '03, 37–48.

- Problem/Gap: • Traditional shortest-path routing in IP networks overloads bottleneck links, increasing delay and jitter.
• Traffic engineering approaches often rely on precomputed paths and prior demand knowledge—unsuited for bursty, dynamic Internet traffic.

- Algorithm/Method: • Potential-Based Routing (PB-routing):
o Assigns a scalar potential to each network element.
o Packets flow along the steepest descent of this potential field.
• PBTA (Potential-Based Traffic-Aware) Algorithm:
o Potential at each node combines:
1. Shortest Path Potential: Linear function of distance to destination—keeps packets goal-directed.
2. Traffic Potential: Derived from queue lengths—pushes packets away from congestion.
o Potentials are computed on a transformed graph where nodes represent links of the original graph, enabling link-based queue modeling.

- Heuristic/Obstacle Handling: • Heuristic: Shortest-path potential acts as the baseline goal-seeking component.
• Obstacle Handling:
o Congested links behave like "hills" in the potential field via traffic potentials.
o Packets naturally follow low-potential (low-congestion) routes.
o Parameter (\alpha) adjusts the balance between path length and congestion avoidance.

- Key Results : • Tested using ns simulations on Waxman, Barabási–Albert, and a real ISP topology.
• Delay & Jitter: Strong reductions compared to shortest-path routing, especially at high loads.
• Packet Loss: PBTA cut packet drops by a factor of 3–5×.
• TCP Compatibility: Very low out-of-order arrivals (0.10%–0.13%).
• Control Overhead: Low, around 2–4% of total traffic.

- Limitations/Open Issues: • Minimal benefit at very low loads (no congestion) and near-saturation loads (no alternatives).
• Less impactful when propagation delay dominates queueing delay (e.g., satellite networks).
• Limited effectiveness in sparsely connected graphs with few alternate routes.

- Relevance to my Research: • Conceptual Model:
o Their potential function provides a direct analogy for your obstacle-aware penalties.
o Similar decomposition can be applied:
[
V(\text{node}) = (1-\alpha) \cdot h(\text{node}, goal) + \alpha \cdot \text{ObstaclePenalty(node)}
]
• Algorithm Design Insight:
o Their hybrid potential (goal-seeking + congestion avoidance) maps closely to combining a heuristic with penalty terms in Bidirectional A*.
• Contrast:
o Focuses on dynamic, traffic-dependent obstacles in IP networks, not static/semi-static obstacles in urban navigation.
o Does not use A* or bidirectional search.
• Additional Inspiration:
o The transformed graph technique is notable if your obstacle penalties depend on edges rather than nodes—may improve the expressiveness of your model.

- Paper File Name : aine2015

- Paper Name: nan

- Citation: · Aine, S., Swaminathan, S., Narayanan, V., Hwang, V., & Likhachev, M. (2015). Multi-Heuristic A.* The International Journal of Robotics Research, 34(1), 1–20.

- Problem/Gap: • Heuristic search algorithms like A* perform poorly when heuristics poorly correlate with actual costs (heuristic depression regions), particularly in high-dimensional or cluttered environments.
• Designing a single admissible and consistent heuristic that captures all problem complexities is challenging.
• Existing approaches that combine multiple heuristics into one lose the individual guiding power of each heuristic.

- Algorithm/Method: • Multi-Heuristic A (MHA):**
o Uses multiple, potentially inadmissible heuristics alongside a single consistent

heuristic.
o Runs multiple searches simultaneously, each guided by a different heuristic, while preserving completeness and sub-optimality guarantees.
• IMHA (Independent MHA):** Each search maintains its own g and h values independently.
• SMHA (Shared MHA):** All searches share a single g value and path information, allowing combination of partial paths to escape depression regions. SMHA* is generally more powerful than IMHA*.

- Heuristic/Obstacle Handling: • Multiple heuristics can be admissible or inadmissible.
• In robotics applications, heuristics often derive from relaxed versions of the problem (ignoring kinodynamic constraints).
• Obstacles are handled implicitly by the cost function and heuristics, which guide the search around obstacles or through narrow passages.

- Key Results : • MHA*, especially SMHA*, outperforms Weighted A* (WA*) and other methods (MPWA*, EES, MHGBFS) in success rate, runtime, and solution cost in high-dimensional robotics domains (e.g., PR2 mobile manipulation, 3D navigation).
• SMHA* achieved ~3× speedup over WA* in indoor environments.
• Less prone to heuristic depression regions, converging faster.
• Provides theoretical guarantees on completeness and sub-optimality even with inadmissible heuristics.

- Limitations/Open Issues: • SMHA* can sometimes be counterproductive if path sharing leads to local minima.
• IMHA* has lower memory overhead and may be preferable when memory is constrained.
• Future work includes anytime versions of MHA*, dynamic heuristic recomputation, and parallel MHA* implementations.

- Relevance to my Research: • Direct Applicability: Addresses A* limitations in complex environments; relevant for enhancing Bidirectional A* with obstacle-aware penalties.
• Integrating Obstacle-Aware Penalties: Penalties can be treated as inadmissible heuristics within the MHA* framework, guiding Bidirectional A* more effectively while maintaining guarantees through a consistent heuristic.
• Bidirectional MHA:* Extending MHA* to Bidirectional A* allows multiple heuristics in both forward and backward searches, influencing search frontiers.
• Handling Depression Regions: Obstacle-aware penalties can help navigate complex urban networks and dynamic traffic-induced heuristic depression regions.

- Paper File Name : applsci-13-08174-v2.pdf

- Paper Name: nan

- Citation: • Almazrouei, K., Kamel, I., & Rabie, T. (2023). Dynamic Obstacle Avoidance and Path Planning through Reinforcement Learning. Applied Sciences, 13(14), 8174.

- Problem/Gap: • Review paper on Reinforcement Learning (RL) applications for dynamic obstacle avoidance (DOA) and path planning (PP) in autonomous mobile robots.
• Highlights the need for a systematic overview of RL in dynamic environments, given computational constraints and long training times.
• Emphasizes challenges faced by traditional algorithms in autonomous navigation of dynamic environments.

- Algorithm/Method: • Reinforcement Learning (RL): Robots learn optimal policies through trial-and-error interactions with the environment.
• Q-learning: Value-based approach maximizing expected future rewards.
• Deep Reinforcement Learning (DRL): Uses neural networks to approximate Q-values for high-dimensional state spaces.
• Hybrid Approaches: RL combined with other techniques, e.g., Artificial Potential Fields (APF) or Probabilistic Roadmaps (PRM).

- Heuristic/Obstacle Handling: • Learning-based Obstacle Avoidance: Robots evaluate expected rewards/penalties based on proximity to obstacles and orientation.
• Dynamic Obstacle Avoidance (DOA): RL excels in environments with moving obstacles.
• Reward-modified DQN: Adjusts reward functions to prevent collisions and encourage obstacle-free paths.
• Black-hole Potential Field: Improved APF combined with RL to avoid local minima.

- Key Results : • RL-based methods effectively handle path planning and dynamic obstacle avoidance.
• DRL succeeds in complex tasks, including indoor navigation and object manipulation.
• Hybrid approaches like QAPF improve learning speed, path length, and smoothness.
• RL enables robots to navigate from start to target without collisions.
• Synthesizes findings from 34 studies (2018–2022), showing increasing research interest in RL for dynamic environments.

- Limitations/Open Issues: • High computational cost for DRL, especially in 3D physics-based simulations.
- Convergence difficulties in complex environments with random initialization.
- Exploration-exploitation trade-offs can limit performance.
- Risk of getting stuck in local minima or producing suboptimal paths.
- Most studies conducted in simulation; real-world deployment remains challenging.
- Limited research on multi-robot coordination and collision avoidance.

- Relevance to my Research: • Dynamic Penalties: Supports using learning-based methods to adjust obstacle-aware penalties adaptively in real-time urban navigation.
- Real-Time Adaptation: RL's capacity to adapt to new environments aligns with enhancing Bidirectional A* for dynamic obstacle handling.
- Hybrid Integration Potential: Combining learning-based penalty adjustments with heuristic search (Bidirectional A*) is a promising approach.
- Handling Dynamic Obstacles: Provides a foundation for implementing adaptive strategies to manage moving obstacles in urban environments.

- Paper File Name : avella2002

- Paper Name: nan

- Citation: · Avella, P., Boccia, M., & Sforza, A. (2002). A penalty function heuristic for the resource constrained shortest path problem. European Journal of Operational Research, 142(2), 221–230.

- Problem/Gap: • Addresses the Resource Constrained Shortest Path Problem (RCSP), an NP-hard problem.
- RCSP requires finding the shortest path between two nodes while respecting limited resource constraints (e.g., time, fuel, money).
- Existing exact methods (Branch-and-Bound, Lagrangean heuristics) are computationally expensive for medium to large networks.

- Algorithm/Method: • Penalty Function Heuristic (PFH): Extends exponential penalty function approaches from continuous linear programming to discrete RCSP.
- Relaxes hard resource constraints by incorporating them into an exponential penalty function.
- Uses a constrained steepest descent algorithm to minimize the penalty over feasible path connectivity.
- Each iteration solves an unconstrained shortest path problem (Dijkstra-like) to find a descent direction.

- Employs a taboo list to avoid cycling and explore a wider solution set.

- Heuristic/Obstacle Handling: • Obstacles are modeled as resource constraint violations; the path is penalized based on how much it exceeds limits.
- Core heuristic: penalty function $\Phi(x)$, aggregating violations across all constraints.
- Iteratively adjusts paths to minimize the penalty, steering the solution toward feasibility.

- Key Results : • Tested on standard OR-library benchmarks and large-scale networks.
- PFH consistently found optimal or near-optimal solutions in fewer than 26 iterations.
- Provided solutions one to two orders of magnitude faster than commercial solvers (XPRESS-MP) with better quality than standard Lagrangean heuristics.
- Example: On a network with 2,560 nodes and 44,160 arcs, PFH found a solution within 9% of optimal in under 3 minutes versus over 2.4 hours for XPRESS-MP.

- Limitations/Open Issues: • Heuristic does not guarantee optimality.
- Solution quality depends on numerical parameters (α, Δ, Maxit).
- Using a k-shortest path for tabooed paths can slow convergence but improves integer solution quality.

- Relevance to my Research: • Penalty-Based Approach: Validates the concept of obstacle-aware penalties in Bidirectional A*, transforming hard constraints (or obstacles) into penalty functions integrated into path cost.
- Integrating Penalties into Path Cost: PFH modifies arc costs iteratively based on penalty gradients; similarly, Bidirectional A* can adjust edge weights to avoid high-penalty regions.
- Heuristic for Urban Navigation: RCSP framework aligns with urban constraints like time windows, vehicle restrictions, or traffic; obstacles can be modeled as resource consumptions.
- Implementation Inspiration: Iterative refinement in PFH provides a template for updating penalty weights and rerunning Bidirectional A* to progressively improve path quality.

- Paper File Name : delarosa2007.pdf

- Paper Name: nan

- Citation: • de la Rosa, T., García Olaya, A., & Borrajo, D. (2007). Using Cases Utility for Heuristic Planning Improvement. In R.O. Weber & M.M. Richter (Eds.), ICCBR 2007, LNAI 4626 (pp. 137–148). Springer-Verlag Berlin Heidelberg.

- Problem/Gap: • Heuristic functions in planners are computationally expensive, consuming most of total planning time.
• Previous Case-Based Reasoning (CBR) approaches did not modify cases based on their utility in solving problems.
• Heuristic planners often generate non-optimal plans; typed sequences extracted from these plans capture only limited object information.
• Retrieval schemes return the first exact match, ignoring potentially better cases.
• Enforced Hill-Climbing (EHC) uses inconsistent heuristics.
• Typed sequences fail to account for goal dependencies, leading to poor performance in domains with strong goal interactions (e.g., Depots domain).

- Algorithm/Method: • CBR Approach: Orders nodes for evaluation in heuristic search to reduce heuristic function calls.
• Dynamic Case Quality Learning: Learns the utility of cases based on validation-phase performance.
• Modified EHC: Successors are evaluated based on recommendations from retrieved cases.
• SAYPHI Architecture: Integrates control knowledge acquisition with a heuristic planner (FF-like).
• Typed Sequences: Abstracted state transition sequences for object types, derived from PDDL domains.
• Utility Measures:
o Step Utility Measure (γ): Frequency of recommending good choices.
o Sequence Utility Measure (λ): Global frequency for sequences, cumulative of step utilities.

- Heuristic/Obstacle Handling: • Focuses on guiding search via learned case utilities rather than modifying the heuristic function directly.
• Utility measures (γ and λ) serve as meta-heuristics to prioritize node/sequences during search.
• Thresholds (μ_step, μ_case) balance exploration (less used cases) and exploitation (more used cases).

- Key Results : • EHC-CBR-Utility reduced heuristic evaluations compared to EHC-CBR across four benchmark domains.
• Satellite domain: 46% reduction in evaluations vs. EHC.
• Satellite and Rovers domains: solved one more problem than EHC.
• Depots domain: EHC-CBR initially worse than EHC, but EHC-CBR-Utility improved

performance.

- Plan length was maintained or slightly improved.
- Main improvement derived from fewer heuristic function calls, increasing planning efficiency.

- Limitations/Open Issues: • Typed sequences only store information about one object, ignoring goal dependencies and relations.

- Future work: extend to numeric domains, incorporate cost functions beyond plan length, handle numeric fluents.
- Heuristic planners generate non-optimal plans; cases are extracted from these, which may limit guidance.

- Relevance to my Research: • Improves heuristic search efficiency via case-based knowledge; this concept can enhance Bidirectional A* search.

- Dynamic learning of case quality and utility measures can help prioritize search directions in complex urban environments.
- Highlights the need to account for interactions between elements, analogous to handling multiple obstacles in navigation.
- Reducing heuristic computation cost aligns with real-time urban navigation requirements.
- Although not directly about obstacle-aware penalties, the utility-based guidance mechanism could be adapted to integrate obstacle-aware penalties into case retrieval and search prioritization.

- Paper File Name : electronics-11-04175.pdf

- Paper Name: nan

- Citation: • Yuan, Y., Shi, Y., Yue, S., Xue, S., Yi, C., & Chen, B. (2022). A Dynamic Obstacle Avoidance Method for AGV Based on Improved Speed Barriers. Electronics, 11(24), 4175.

- Problem/Gap: • Traditional adaptive speed obstacle methods can cause AGVs to make excessive turns near obstacles due to exponential expansion of the collision cone, risking instability or collisions.

- Candidate velocities may all fall inside the collision cone, leaving no feasible optimal speed.
- Dynamic uncertainties—like unpredictable obstacle motion, sensor errors, and wheel slippage—make accurate trajectory prediction difficult.

- Algorithm/Method: • Improved Speed Obstacle Algorithm: Enhances the classical Velocity Obstacle (VO) method.

• Kalman Filtering: Predicts future positions of dynamic obstacles.

• Forward Simulation & Speed Obstacle Buffer: Constructs a collision cone in velocity space based on predicted positions rather than current ones.

• Comprehensive Evaluation Index: Scores candidate velocities with a weighted sum of safety and efficiency, selecting the highest-rated speed.

- Heuristic/Obstacle Handling: • Obstacle Position Prediction: Uses a Kalman filter to estimate obstacle positions at the next time step.

• Speed Obstacle Buffer: Collision cones are based on predicted future positions, allowing earlier and smoother reactions.

• Safety/Efficiency Objective Function:

o (G = \alpha \cdot \text{dist} + \beta \cdot \text{avo})

 Safety (dist): Distance between AGV's velocity vector and obstacle center; larger distance = safer.

 Efficiency (avo): Angle between candidate velocity and cone boundary; smaller angle = more efficient.

o The algorithm selects the candidate velocity maximizing (G), balancing safety and efficiency.

- Key Results : • Success rate improved by 11% over traditional adaptive speed obstacle method under skid conditions (98% vs. 87%).

• Maintains greater safety distances (shortest distance 0.6992 m vs. 0.6755 m adaptive, 0.5038 m VO).

• Smaller turning angles lead to more stable trajectories.

• Path length and time slightly longer than traditional VO, but comparable to adaptive method, indicating a trade-off between efficiency and safety.

- Limitations/Open Issues: • Models AGV and obstacles as particles, not accounting for real vehicle shapes and kinematics.

• Kalman filter effectiveness depends on motion model accuracy; erratic obstacles reduce prediction accuracy.

• Objective function weights ((\alpha, \beta)) are experimentally set, may not generalize optimally.

- Relevance to my Research: • Predictive Obstacle Modeling: Using predicted positions to adjust paths proactively can enhance Bidirectional A* for real-time navigation.

• Obstacle-Aware Penalty Concept: The "speed obstacle buffer" can be translated into dynamic node penalties in A*, increasing costs for nodes near predicted collision zones.

• Multi-Objective Heuristic: Safety and efficiency balancing inspires a heuristic that

penalizes paths based on risk as well as length.

- Robustness Testing: Simulating uncertainty (e.g., skidding) highlights the need to evaluate my Bidirectional A* under non-ideal, realistic conditions.

- Paper File Name : efficient-modified-bidirectional-a--algorithm-for-optimal-routef.pdf

- Paper Name: nan

- Citation: • Whangbo, T.-K. (2007). Efficient Modified Bidirectional A* Algorithm for Optimal Route-Finding. In IEA/AIE 2007 (pp. 344–353). Springer, Berlin, Heidelberg.

- Problem/Gap: • Traditional Bidirectional A* (BA*) is faster than unidirectional A* but cannot guarantee optimality if the search stops when the two frontiers meet.
- Guaranteeing optimality requires additional termination conditions, which can make BA* slower than unidirectional A* in the worst case.
- The goal is to design a modified BA* that maintains efficiency and guarantees optimality.

- Algorithm/Method: • Modified Bidirectional A (MBA*)**: Guarantees optimality and reduces search time relative to conventional BA.
- Performs simultaneous forward (start→goal) and backward (goal→start) search.
- Introduces a new optimality condition using:
  - Lmin (minimum path cost found so far).
  - Heuristic estimations of frontier nodes relative to a “separating line” between forward/backward searches.
- This condition allows earlier termination without sacrificing optimality.
- Avoids redundant or repetitive searching.

- Heuristic/Obstacle Handling: • Uses standard A* heuristic: $f(n) = g(n) + h(n)$.
- No explicit obstacle-penalty heuristic; obstacles are assumed to be static and encoded in graph edges and costs.
- Focus is on optimizing the search procedure rather than modeling obstacle dynamics.

- Key Results : • Tested on real Seoul urban road data (10,590 nodes; 22,874 links).
- MBA* expanded ~50% fewer nodes than unidirectional A* and ~33% fewer nodes than conventional BA*.
- Running time reduced proportionally to the reduction in node expansions.
- Always guarantees an optimal route.

- Limitations/Open Issues: • Derived from static road networks; requires extension to handle traffic, road status, weather, etc.
• Does not address dynamic environments or moving obstacles.

- Relevance to my Research: • Directly relevant as it optimizes Bidirectional A*, the core of my work.
• The mention of integrating conditions like traffic or road status aligns with my goal of adding obstacle-aware penalties to the cost or heuristic functions.
• MBA*'s optimality guarantee provides a foundation for ensuring that my penalty-augmented heuristic does not break optimality, or for adapting the termination criteria accordingly.
• Provides an empirical benchmark for comparing the performance gains of my enhanced Bidirectional A*.

- Paper File Name : elsisi2018.pdf

- Paper Name: nan

- Citation: • Elsisi, M. (2018). Future search algorithm for optimization. Evolutionary Intelligence, 11(3–4), 147–158.

- Problem/Gap: • Focuses on general optimization and the difficulty of finding global optima efficiently.
• Existing heuristic algorithms (GA, PSO, etc.) may converge slowly when starting far from good solutions.
• Many standard meta-heuristics can get trapped in local optima due to relying mostly on either global-best or local-best information.
• Goal: design a heuristic algorithm that converges fast, avoids local minima, and remains computationally simple.

- Algorithm/Method: • Future Search Algorithm (FSA): A new meta-heuristic inspired by human “search for a better future.”
• Combines local exploration (interaction among “people” within a country) and global exploration (interaction among historically best candidates across countries).
• Periodically reinitializes or updates “initial positions” to expand exploration and prevent stagnation.
• Uses lightweight mathematical update rules → low computational overhead.
• Iterative process meant to strike a balance between exploration (global) and exploitation (local).

- Heuristic/Obstacle Handling: • Not about spatial obstacles; “obstacles” correspond to local optima in the objective landscape.
• Heuristic mechanisms involve balancing local and global search to avoid becoming trapped in poor solutions.
• Key technique: dynamic repositioning of candidate solutions to increase the diversity of search.

- Key Results : • Evaluated on 23 common unimodal and multimodal benchmark functions.
• Outperformed GA, PSO, GSA, FFA, and LSA on nearly all test functions.
• Demonstrated faster convergence and strong ability to avoid local minima.
• Particularly effective on multimodal functions requiring extensive exploration.

- Limitations/Open Issues: • Designed for general function optimization, not spatial or graph-based search.
• Runtime and scalability for real-world pathfinding contexts are not analyzed.
• “Human behavior” analogy is conceptual; algorithm must be significantly adapted for navigation or A* contexts.
• Does not model dynamic environments or sequential decision-making.

- Relevance to my Research: • Parameter Optimization for Penalties: FSA could optimize the weighting parameters in my obstacle-aware penalty function instead of relying on manual tuning.
• Avoiding Local Optima: If my penalty landscape becomes complex (e.g., penalties causing the search to settle on suboptimal detours), an FSA-like meta-heuristic could help discover global penalty configurations that yield better navigation performance.
• Adaptive Mechanisms: FSA’s blend of local and global adaptation suggests ways to dynamically adapt search strategy or penalty strength in Bidirectional A*.
• Indirect but Useful: While FSA is not directly applicable to pathfinding, its optimization principles can be incorporated into the meta-level of my algorithm—for tuning penalty parameters or guiding adaptive decision-making.

- Paper File Name : fleischmann2004.pdf

- Paper Name: nan

- Citation: Fleischmann, B., Gnutzmann, S., & Sandvoß, E. (2004). Dynamic Vehicle Routing Based on Online Traffic Information. Transportation Science, 38(4), 420–433.

- Problem/Gap: • Most Vehicle Routing Problem (VRP) models assume static travel times, which is unrealistic in urban environments with constantly changing traffic.
• Conventional VRP solutions do not fully integrate real-time traffic information or accommodate dynamic order arrivals.
• Dynamic shortest path computation for every possible start time and destination is computationally expensive.
• The volume of dynamic traffic information requires an efficient filtering mechanism to avoid processing irrelevant updates.

- Algorithm/Method: • Dynamic Vehicle Routing System Architecture
o Traffic Observation System: Collects real-time traffic data and detects deviations.
o Order & Fleet Management System (OFMS): Manages incoming orders and vehicle states.
o Planning System: Computes routing decisions using updated travel times.
• Integration of Online Traffic Information
o Uses real-time travel time data from a traffic management center.
o Incorporates incident-based updates (accidents, congestion spikes).
o Includes travel time forecasts.
• Modified Dijkstra Algorithm
o Adapted for time-dependent travel times.
o Operates on arcs instead of nodes, improving efficiency for dynamic updates.
• Planning Procedures with varying time horizons:
o Assignment Rules (AR): Simple heuristic rules for assigning new jobs to idle vehicles.
o Assignment Algorithm (AS / AP): Optimal order-to-vehicle assignment using the LAPJV algorithm (Hungarian method variant).
o Insertion Algorithms (INS): Insert new orders into existing tours, followed by resequencing using Or-opt.
• Cost Criteria
o Objective functions include costs for:
 Empty distance
 Loaded distance
 Waiting
 Delay (quadratic penalty)
o These shape the routing behavior toward lower delays and higher efficiency.
• Path Monitoring
o Observation system monitors routed paths for travel-time deviations and triggers recalculation when significant.

- Heuristic/Obstacle Handling: • Time-Dependent Travel Times
o Real-time traffic updates function as a heuristic signal, dynamically altering arc costs.

o Congestion or incidents act as implicit obstacles via increased traversal costs.
• Quadratic Cost Penalties
o Waiting and delay penalties serve as soft constraints guiding routing decisions.
• Filtering Mechanism
o Reduces computational load by keeping only relevant traffic updates for routing decisions.

- Key Results : • The Assignment Algorithm (AS) outperformed Assignment Rules (AR) and Insertion methods (INS).
o Example (Dataset C):
 AS → 0 late orders, 5.3 min avg. delay
 INS1 → 131 late orders, 21.5 min avg. delay
• Computation times were low, even on outdated hardware (350 MHz PC):
o <350 ms at order arrival
o ~25 ms at order completion
• The use of dynamic travel times significantly improved routing performance and reliability.

- Limitations/Open Issues: • Assumes single-load pickup and delivery, simplifying real capacity constraints.
• No mechanism for order rejection (e.g., due to insufficient capacity or time slack).
• LAPJV is solved mostly from scratch; incremental updates are efficient only when traffic changes are small.
• Simulations rely on historical rather than live real-time tests.
• Does not account for dynamic moving obstacles—only time-dependent travel cost changes.

- Relevance to my Research: • Demonstrates the importance of real-time, time-dependent information for urban navigation—core to my project.
• The modified Dijkstra approach provides insight into how Bidirectional A* can incorporate dynamic, obstacle-aware penalties.
• The paper's quadratic delay and waiting penalties parallel my idea of using obstacle-aware penalty functions—showing how soft penalties can guide route selection.
• Its methods for managing computational load are directly relevant for real-time performance in my algorithm.
• While the paper treats traffic as a dynamic “obstacle,” its approach provides a framework for integrating dynamic environmental data—aligning with my goal of embedding explicit dynamic obstacle predictions into the Bidirectional A* heuristic.

- Paper File Name : FULLTEXT02.pdf

- Paper Name: nan

- Citation: Jern, S., & Salomonsson, J. (2024). Multi-Target Pathfinding: Evaluating A-star Versus BFS (Bachelor Thesis). Malmö University.

- Problem/Gap: - Compares the core A* and Breadth-First Search (BFS) algorithms in multi-target scenarios, specifically when complex implementations are not preferred or possible.
- Addresses the question of which algorithm is more efficient under varying conditions (number of targets, obstacle density, grid size) for single-source, multi-target pathfinding.
- Highlights that A* typically requires a separate search for each target, while BFS can find paths to multiple targets in one search.

- Algorithm/Method: - A* Algorithm: Uses Manhattan Distance as its heuristic function. Performs a separate search for each target.
- Breadth-First Search (BFS): Explores uniformly in all directions from the start node. Can find paths to multiple targets in a single search.
- Simulation Framework: A Java-based framework was developed to systematically simulate these algorithms across various scenarios (varying number of targets, grid sizes, and obstacle densities) in a static, grid-based environment with 4-way movement.

- Heuristic/Obstacle Handling: - A* Heuristic: Manhattan Distance is used to estimate the cost from a node to the target, guiding the search.
- Obstacles: Represented as non-traversable tiles on a grid, randomly placed. The obstacle density is a key variable in the simulations.
- The environment is static; obstacles and targets do not move.

- Key Results : - Single Target: A* generally outperforms BFS in execution time and nodes traversed, especially in larger grids and lower to moderate obstacle densities. BFS can outperform A* in smaller grids or very high obstacle densities.
- Multiple Targets:
  - BFS becomes more effective than A* in scenarios with a higher number of targets and smaller search spaces, as it can find paths to multiple targets in one search.

  - A* performance degrades significantly with an increasing number of targets due to the need for separate searches for each target.
  - On large grids (512x512), A* loses its efficiency advantage over BFS when the number of targets reaches approximately 10.
- Obstacle Density: A* is less affected by low to moderate obstacle density. At higher densities (around 30%), BFS can outperform A* in execution time.
- Grid Size: A* has a notable advantage over BFS in larger grids for single-target scenarios. BFS struggles with larger grids due to increased node traversal.

- Limitations/Open Issues: - Simplified Context: Assumes unweighted graphs, 4-way movement,
shared starting position, and static positioning of targets and obstacles.
- Heuristic Choice: Only Manhattan Distance was used for A*. Exploring other heuristics could provide further insights.
- Dynamic Environments: The study explicitly states that future research could involve dynamic environments where obstacles and targets move or change states, which would introduce more complexity. This is a direct gap for my research.
- Bidirectional Search: The thesis does not implement or compare bidirectional versions of A* or BFS, which are often more efficient.

- Relevance to my Research: - Comparative Analysis Baseline: This paper provides a valuable
comparative analysis of core A* and BFS, which are foundational to pathfinding. It offers insights into their performance under varying conditions, which can serve as a baseline for understanding the behavior of Bidirectional A* in urban navigation.
- Impact of Obstacle Density: The findings on how obstacle density affects performance are directly relevant. My "obstacle-aware penalties" will essentially increase the perceived "density" or cost of certain areas, and this paper helps understand how such changes might impact A* performance relative to other methods.
- Multi-Target vs. Single-Target: The distinction between single-target and multi-target scenarios and their impact on A* efficiency is important. While my research focuses on a single destination, the concept of "obstacle-aware penalties" could be

seen as creating multiple "undesirable targets" to avoid, which
might influence the choice of algorithm or its configuration.
- Future Research Alignment: The explicit mention of dynamic
environments as a future research direction directly aligns with my research on "Enhancing Bidirectional A* for Real-Time Urban Navigation using obstacle-aware penalties." This paper
provides a clear justification for the need for such research.

- Paper File Name : garvin2010.pdf

- Paper Name: nan

- Citation: Garvin, B. J., Cohen, M. B., & Dwyer, M. B. (2011). Evaluating

improvements to a meta-heuristic search for constrained interaction

testing. Empirical Software Engineering, 16(1), 61-102.

- Problem/Gap: - Combinatorial Interaction Testing (CIT) for highly-configurable
systems is challenged by feature constraints (some features cannot
coexist).
- Greedy algorithms for CIT generate larger samples (more
configurations to test) when constraints are present.
- Meta-heuristic algorithms (like simulated annealing) for CIT have
long run times when feature constraints are present.
- Neither greedy nor meta-heuristic approaches are cost-effective when
both constraints and the cost of testing configurations are
important.
- Simulated annealing's search can get stuck in local optima or
infeasible states due to constraints.
- The binary search in the outer loop of simulated annealing makes
faulty assumptions about the inner search's ability to find a
solution for a given array size.

- Algorithm/Method: - Reformulated Simulated Annealing: Modified an existing simulated annealing algorithm to more efficiently incorporate constraints for
Constrained CIT (CCIT).
- Primary Modifications:
  - t-Set Replacement: Instead of changing individual symbols, the
  transformation function writes an entire missing t-set to a row,
  making the search more goal-oriented and less prone to
  constraint violations.
  - One-Sided Narrowing: Modifies the outer binary search to only

improve the upper bound, abandoning the assumption that a failed inner search precludes a solution of a given size. This allows
the algorithm to revisit array sizes.
- Refinements: - Row Replacement: A meta-heuristic that detects when the search
gets stuck and offers an additional way to escape by replacing
an entire row with a new, valid one.
- SAT History: The algorithm remembers symbols rejected by
constraints and avoids trying them again.
- Row Sorting: Valuable rows (those covering difficult t-sets) are
moved to earlier locations in the array to improve search
efficiency.
- Iteration Bounding: Adaptive approach to setting iteration
limits for the inner search, adjusting based on search progress.
- Informed Partitions: The algorithm estimates the best choice for
the next array size (N) in the outer search based on the
difficulty of previous inner searches.
- Bounds Revision: Adds checks to the outer search to adjust lower
and upper bounds for array size, especially in constrained
problems where initial estimates might be inaccurate.
- SAT Solver Integration: Tightly integrates an off-the-shelf SAT

solver (MiniSAT) to check constraint satisfaction before accepting

moves.

- Heuristic/Obstacle Handling: - The paper focuses on optimizing a meta-heuristic search (simulated
annealing) for combinatorial interaction testing under constraints.
The "obstacles" here are the constraints themselves, which limit the
search space and can trap the algorithm.
- The modifications (t-set replacement, row replacement, SAT history)
act as heuristics to navigate this constrained search space more
effectively, avoiding infeasible regions and guiding the search
towards valid solutions.
- The fitness function (number of yet-to-be-covered t-sets) guides the
simulated annealing towards a solution.
- The cooling schedule and probabilistic acceptance of
fitness-worsening moves are inherent heuristics of simulated
annealing, balancing exploration and exploitation.

- Key Results : - The optimized simulated annealing (ALL version) reduced run time by
a factor of 90 and achieved the same coverage objectives with even
fewer system configurations compared to the base algorithm.
- On constrained problems, the updated simulated annealing produced

CIT samples with an average of 25% fewer configurations than greedy algorithms.
- The reformulated simulated annealing outperformed greedy algorithms on real-world problems when the per-configuration testing time exceeded a break-even point (e.g., 21 seconds on average).
- The modifications improved the robustness of simulated annealing, making its results as repeatable as greedy algorithms and more resilient to input variations.
- The ALL implementation was consistently faster than the CONTROL version on unconstrained problems by a factor of over 300.

- Limitations/Open Issues: - Limitations / Open Issues:
  - The study's benchmarks (real-world and synthetic) may not fully represent all configurable software systems.
  - The full parameter space for simulated annealing was not explored due to prohibitive computational cost, potentially biasing results.
  - The paper acknowledges that the effectiveness of some refinements (e.g., informed partitions) needs further evaluation.
  - The break-even point calculation assumes equal mean per-configuration test times, which might not always hold true.

- Relevance to my Research: - This paper offers valuable insights into optimizing meta-heuristic
search algorithms for constrained problems, which can be directly applied to enhancing Bidirectional A* in urban navigation with obstacle-aware penalties.
- The concept of "t-set replacement" and "row replacement" to escape local optima and navigate a constrained search space can inspire similar strategies in my Bidirectional A* implementation, especially when dealing with complex obstacle configurations that might lead to dead ends or suboptimal paths.

The use of a SAT solver to enforce constraints is analogous to how I might integrate obstacle avoidance rules directly into the

pathfinding process, ensuring that generated paths are always valid.
* The emphasis on balancing computational time and solution quality

(sample size/path length) is highly relevant to real-time urban

navigation, where both speed and optimal path are crucial.

* The paper's detailed analysis of how different algorithmic

modifications impact performance and robustness provides a framework
for evaluating the effectiveness of my proposed enhancements to

Bidirectional A*.*

- Paper File Name : habib2007.pdf

- Paper Name: nan

- Citation: Habib, M. K. (2007). Real Time Mapping and Dynamic Navigation
for Mobile Robots. International Journal of Advanced Robotic
Systems, 4(3), 323-338.

- Problem/Gap: - Addresses the challenges of real-time mapping and dynamic
navigation for mobile robots in unknown and dynamic
environments.
- Focuses on the need for robust and reliable navigation systems
that can handle moving obstacles, sensor uncertainty, and
real-time constraints.
- Highlights the limitations of traditional methods in dynamic
environments and the need for adaptive representations and
models.

- Algorithm/Method: - Incremental Grid-Based Mapping: Uses occupancy grids to
represent the environment, incrementally building and
maintaining the map.
- Modified Histogrammic In-Motion Mapping (HIMM): An algorithm
suitable for real-time obstacle detection and avoidance,
integrated with the occupancy grid concept.
- Fusion of Multiple Ultrasonic Sensory Information: Combines
data from multiple sensors to improve accuracy and robustness.
- Parallel and Distributed Framework: Mapping and obstacle
avoidance modules are deployed in parallel to ensure real-time
operation with limited resources and promote extensibility.

- Heuristic/Obstacle Handling: - Occupancy Grids: Represent the environment as a grid
where each
cell has a probability of being occupied by an obstacle. This
is the core mechanism for representing and detecting obstacles.
- Histogrammic In-Motion Mapping (HIMM): Processes sensor data
(ultrasonic range readings) to update the certainty values of
grid cells, effectively identifying obstacles and free space.
- Dynamic Obstacle Avoidance: The system is designed to react to

unexpected events and change course, learning from experiences
to improve behavior.
- Penalty/Cost (Implicit): While not explicitly stated as
"penalties," the system's goal is to avoid collisions, implying
that paths through occupied or uncertain cells would have a
higher implicit cost or be avoided.

- Key Results : - Simulation-based experiments demonstrated the validity of the
developed mapping and obstacle avoidance approach.
- The system can map the environment, avoid obstacles, and move
towards its target successfully in real-time.
- The incremental grid-based mapping and HIMM approach proved
effective for real-time operation.

- Limitations/Open Issues: - The paper focuses on local, reactive obstacle avoidance and
mapping, rather than global path planning.
- The approach is demonstrated through simulations; real-world
deployment might introduce additional complexities.
- The paper does not explicitly integrate with global pathfinding
algorithms like A* or Bidirectional A*.
- The discussion on "learning" is more about adapting to sensor
data and updating maps rather than learning optimal pathfinding
strategies.

- Relevance to my Research: - Real-time Obstacle Representation: This paper provides a
concrete method (occupancy grids with HIMM) for representing
and updating dynamic obstacle information in real-time. This is
highly relevant for generating the "obstacle-aware penalties"
in my Bidirectional A* algorithm.
- Dynamic Environment Handling: The focus on dynamic environments
and real-time operation directly aligns with my research goals
for urban navigation. The techniques for handling sensor data
and updating environmental models can inform how my system
perceives and reacts to dynamic obstacles.
- Integration with Global Planner: While this paper focuses on
local avoidance, its methods can be integrated with a global
path planner. The occupancy grid data, updated in real-time,
can be used to dynamically adjust the costs (penalties) of
edges in the graph used by Bidirectional A*, thus making the
global path planner obstacle-aware.
- Foundation for Penalties: The concept of "certainty values" in
occupancy grids can be directly translated into the magnitude
of "obstacle-aware penalties." Higher certainty of an occupied

cell would lead to a higher penalty for traversing through or near it.

- Paper File Name : hu2016.pdf

- Paper Name: nan

- Citation: Hu, L., Yang, J., & Huang, J. (2016). The real-time shortest path algorithm with a consideration of traffic-light. Journal of Intelligent & Fuzzy Systems, 31(5), 2403-2410.

- Problem/Gap: - In urban areas, delay due to traffic lights is a significant factor that must be considered when searching for the optimal path.
- The shortest physical path is not necessarily the shortest time-cost path.
- Existing algorithms often focus only on minimizing time (Least Expected Time - LET), but drivers often need to consider both time and length.
- Calculating paths considering both time and length, which are different physical measures, can make algorithms complex.

- Algorithm/Method: - Improved A* Algorithm: The paper improves the standard A* algorithm by modifying its heuristic function.
- Translation Module: The core of the algorithm is a "translation module" that converts the time delay from traffic lights into an equivalent spatial length. This combines time and length into a single, unified measure.
- Modified Heuristic Function: The new heuristic function h'(N) is the sum of the original heuristic h(N) (estimated distance to target) and the translated distance from traffic light delay d(N). The evaluation function becomes $f(N) = g(N) + h(N) + d(N)$.

- Heuristic/Obstacle Handling: -
  - Traffic Light Delay as an Obstacle: The primary "obstacle" is the delay caused by traffic lights.
  - Delay-to-Distance Translation: The waiting time at a traffic light (tw) is translated into an equivalent distance d(N) using the formula $d(N) = v * tw$, where v is the vehicle's speed.
  - Vehicle Speed Modeling: Vehicle speed v is modeled using a Gaussian distribution $N(\mu, \sigma^2)$, with real-time average speed μ and standard deviation σ data provided by the Beijing Municipal Commission of Transport (BMCT), refreshed every five minutes.

- Waiting Time Estimation: The waiting time tw at an intersection is estimated based on the traffic light cycle time (T), the vehicle's arrival time (t), and the probability of turning in a specific direction. The average waiting time is calculated as a weighted average of the waiting times for different turning possibilities.

- Key Results : - A simulation on a simplified map showed that the improved algorithm found a path that was physically 10% longer but resulted in a 5.8% reduction in travel time compared to the standard A* algorithm.
- The calculated "equivalent length" (Ls) for the path found by the improved algorithm was 4% shorter than the path found by the standard A* algorithm.
- The improved algorithm successfully found a route with fewer traffic lights and more right turns, which are generally faster maneuvers.

- Limitations/Open Issues: The precision of the algorithm may decrease as the path gets longer

because the real-time speed data changes over time, and the initial

speed estimate becomes less accurate for later parts of the journey.
* The waiting time at intersections is an approximation calculated at

the beginning of the path search, which is a simplification and

could be inaccurate.

* The algorithm still has points for future improvement, as

acknowledged by the authors.

- Relevance to my Research: - This paper directly addresses the integration of real-world urban
navigation constraints (traffic lights) into a classic pathfinding algorithm (A*), which is central to my research.
- The core concept of a "translation module" to convert a time-based penalty (traffic light delay) into a distance-based cost for the A* heuristic is a powerful and directly applicable idea for my own "obstacle-aware penalties."
- My Bidirectional A* algorithm can adopt a similar methodology: translate the risk or delay from dynamic obstacles into an additional cost for the heuristic function, influencing the search direction in both the forward and backward searches.
- The use of real-time, probabilistic data (Gaussian distribution for

speed) is a good example of how to handle the uncertainty inherent in urban environments, inspiring a similar approach for modeling obstacle behavior in my research.
- The paper's limitation regarding the decreasing precision over longer paths highlights a challenge I must also address. Bidirectional search might inherently mitigate this by meeting in the middle, reducing the "look-ahead" distance for both searches.

- Paper File Name : ICJE-6-1-243-248.pdf

- Paper Name: nan

- Citation: Shen, Y. (2020). Optimization of Urban Logistics Distribution path

under dynamic Traffic Network. International Core Journal of

Engineering, 6(1), 243-248.

- Problem/Gap: - Traditional Vehicle Routing Problems (VRP) often assume static driving speeds, which is unrealistic in dynamic urban environments with traffic congestion (rush hour, accidents).
- This leads to distribution vehicles getting stuck in crowded roads, increasing actual operation costs and cost errors.
- There is a lack of consideration for "hard time windows" in dynamic VRP research, where failure to provide service within the window leads to customer refusal.

- Algorithm/Method: - Logistics Distribution Path Optimization Model: Established for hard time windows under dynamic road networks.
- Objective Function: Minimize total distribution cost, which includes running cost of the vehicle and the cost of enabling the delivery vehicle.
- Dynamic Running Speed Calculation: Vehicle speed is calculated across different time periods to account for varying traffic conditions (peak/off-peak hours).
- Genetic Algorithm (GA): Used to solve the optimization model.
  - Natural Coding: Chromosomes represent initial feasible distribution routes.
  - Fitness Calculation: Higher fitness corresponds to lower cost.
  - Selection, Crossover, Mutation: Standard GA operations are applied to evolve new species groups (optimal solutions).

- Heuristic/Obstacle Handling: - Dynamic Travel Times: The model implicitly handles "obstacles" like

traffic congestion by calculating vehicle running speeds that vary with time periods (e.g., peak vs. off-peak). This means that congested roads are effectively penalized by slower speeds, increasing their cost in the pathfinding process.
- Hard Time Windows: These act as strict constraints. If a vehicle cannot arrive within the specified time window, the solution is considered infeasible or incurs a very high penalty (customer refusal). This forces the algorithm to find paths that avoid "temporal obstacles."
- Genetic Algorithm Heuristics: The GA itself is a meta-heuristic search algorithm that explores the solution space to find optimal or near-optimal paths by simulating natural evolution.

- Key Results : - The model was tested using the Solomon R101 calculation example.
- The genetic algorithm successfully solved the distribution path problem, considering dynamic road networks and hard time windows.
- The results demonstrate the effectiveness of the model in optimizing logistics distribution paths under complex urban conditions.
- The paper shows that the model is effective in handling variable information in dynamic road networks.

- Limitations/Open Issues: The paper states that "there are too many variable information in

the dynamic road network, there are many factors that are not taken

into account," suggesting that the model could be further refined.

* Future work includes optimizing logistics distribution paths for

different road levels, multiple vehicle distribution, and customer

random demand.

* The paper does not provide quantitative comparisons with other

algorithms or detailed performance metrics (e.g., computation time,

solution quality improvement percentage).

- Relevance to my Research: - This paper's focus on dynamic urban networks and hard time windows
is highly relevant to my research on real-time urban navigation.
- The approach of incorporating time-varying speeds to account for

traffic congestion (a form of "soft obstacle") can be adapted to my Bidirectional A* algorithm. I can use dynamic speed profiles or cost functions that change based on predicted traffic conditions.
- The concept of "hard time windows" as strict constraints can be integrated into my pathfinding, where certain areas or routes might be temporarily inaccessible or highly penalized during specific times due to dynamic obstacles.
- While using a Genetic Algorithm, the paper's problem formulation (minimizing cost under dynamic conditions and time windows) is directly applicable to defining the objective function and constraints for my Bidirectional A* algorithm.
- The acknowledgment of "many factors not taken into account" in dynamic road networks inspires me to consider a broader range of dynamic obstacle characteristics and their impact on pathfinding.

- Paper File Name : isa2015.pdf

- Paper Name: nan

- Citation: Isa, N., Mohamed, A., & Yusoff, M. (2015). Implementation of Dynamic Traffic Routing for Traffic Congestion: A Review. In SCDS 2015 (pp. 174-186). Springer, Singapore.

- Problem/Gap: - This is a review paper focusing on dynamic traffic routing as a solution to traffic congestion, particularly in urban areas.
- It addresses the need for efficient methods to disperse traffic congestion, both recurrent (e.g., peak hours) and non-recurrent (e.g., accidents, road closures).
- Highlights the limitations of traditional routing that doesn't adapt to real-time traffic conditions.

- Algorithm/Method: - Dynamic Traffic Routing: The core concept reviewed, which involves re-routing vehicles based on real-time traffic information.
- Online vs. Offline Routing:
  - Offline: Routes are pre-planned; can be robust (using predicted data) but assumes 100% correct predictions.
  - Online: Algorithms respond to real-time changes, re-calculating routes based on current traffic conditions. This is crucial for dynamic environments.
- Update Strategies: Discusses time-based updates (discrete or interval) and node/intersection-based updates (when vehicles arrive at an intersection).

- Optimization Methods: Mentions various algorithms used in dynamic routing, including Dijkstra, Ant Colony Optimization, Genetic Algorithms, and Brownian Agent models.

- Heuristic/Obstacle Handling: - Traffic Density/Flow: These are the primary "obstacle-aware
penalties" in this context. High traffic density or low traffic flow on a road segment indicates congestion, which should be avoided.
- Stochastic Variables: Traffic cost, route density, traffic demand, vehicle speed, and traffic flow are treated as stochastic variables that change dynamically.
- Re-routing: The mechanism to avoid "obstacles" (congested areas) by calculating new, less congested routes.

- Key Results : - Dynamic traffic routing is an important method for optimizing traffic congestion relief.
- Online routing, which adapts to real-time changes, is essential for dynamic environments.
- Various optimization algorithms have been applied, with different update strategies for stochastic traffic variables.
- The paper synthesizes current research on how dynamic traffic routing is implemented and its limitations.

- Limitations/Open Issues: - Computational Cost of Re-routing: Re-routing all vehicles or for multiple origins/destinations can be computationally expensive.
- Focus on Single Vehicle: Many studies focus on guiding a single vehicle, rather than optimizing for the entire network or multiple vehicles simultaneously.
- Non-recurrent Congestion: Handling unexpected events like road closures requires different strategies (e.g., detour plans) than recurrent congestion.
- Data Accuracy: The effectiveness of dynamic routing heavily relies on accurate real-time and predicted traffic data.

- Relevance to my Research: - Direct Problem Alignment: This paper is highly relevant as it directly addresses the problem of real-time urban navigation and dynamic traffic congestion, which is the core of my research.
- "Obstacle-Aware Penalties" as Traffic Variables: The concept of using traffic density, flow, and travel time as dynamic variables to guide routing directly translates to my "obstacle-aware penalties." I can use these metrics to dynamically adjust the costs of edges in my Bidirectional A*

algorithm.
- Online Routing Strategy: The emphasis on online routing and real-time updates reinforces the need for my Bidirectional A* algorithm to be highly efficient and capable of rapid re-computation in dynamic urban environments.
- Integration with A*: The paper explicitly mentions Dijkstra and A* as optimization methods for dynamic routing, providing a clear pathway for integrating my enhanced Bidirectional A* into such systems.
- Future Research Direction: The open issues regarding computational cost for network-wide re-routing and handling non-recurrent congestion provide clear avenues for my research to contribute by developing a more efficient and adaptive Bidirectional A* with obstacle-aware penalties.

- Paper File Name : janson1991.pdf

- Paper Name: nan

- Citation: Janson, B. N. (1991). Dynamic traffic assignment for urban road

networks. Transportation Research Part B: Methodological, 25(2-3),

143-161.

- Problem/Gap: - Simulating traffic conditions on urban highways during peak periods requires dynamic traffic assignment (DTA) models.
- Existing static user-equilibrium assignment (SUE) procedures and dynamic models are limited by network size restrictions or make steady-state assumptions.
- Previous DTA formulations often do not ensure temporally continuous flows, leading to unrealistic traffic patterns.
- The dynamic user-equilibrium (DUE) problem is complex due to nonlinear flow conservation constraints and temporal flow continuity constraints, making it difficult to solve with traditional linear combination methods.
- Linear combination methods (e.g., Frank-Wolfe) can create temporally discontinuous flows when applied to DUE.

- Algorithm/Method: - Dynamic Traffic Assignment (DTA) Heuristic: Developed to generate approximate solutions to the Dynamic User-Equilibrium (DUE) problem for large networks efficiently.
- Nonlinear Programming Formulation: The DUE problem is formulated as

a nonlinear program with multiple trip origins and destinations, extending SUE to include temporal aspects.
- Shortest Path Trees: The DTA procedure finds and loads shortest path trees based on projected link volumes in future time intervals.
- Link Volume Projection: Future link volumes are estimated using a weighted combination of current link volumes and ratios of future to current travel demands (Equation 16). This allows the algorithm to anticipate future congestion.
- Incremental Assignment: Trips departing in each time interval are assigned incrementally to complete paths, with link use intervals based on shortest path travel times.
- Random Origin Selection: Trips are assigned from origins chosen in a geographically random order to reduce random variability in link volumes.

- Heuristic/Obstacle Handling: - Dynamic Link Impedance Functions: Travel times (impedances) on links
are not static but are functions of link volume, reflecting congestion as a dynamic "obstacle."
- Projected Link Volumes: The algorithm uses projected future link volumes to anticipate congestion, effectively acting as an obstacle-aware mechanism that looks ahead in time.
- Temporally Continuous Flows: Constraints are introduced to ensure that vehicle flows are continuous over time, preventing unrealistic "teleportation" or discontinuous paths.
- DTA as a Heuristic: The DTA procedure itself is a heuristic designed to approximate DUE conditions efficiently, rather than finding an exact, convergent solution.

- Key Results : DTA successfully produced assignments that approximately satisfy

dynamic user-equilibrium conditions.

* Compared favorably with the Frank-Wolfe (F-W) method for static

assignments, producing similar SUE solutions with manageable memory

requirements for large networks.

* DTA required significantly less memory than linear combination

methods for DUE (four times less).

* For the Sioux Falls network, DTA produced static assignments with 3%

average percent link volume variation between time intervals and

1.51% impedance gap (GAP1).

* For the Pittsburgh network with dynamic travel demands, GAP1 was

0.483%, slightly higher than for static assignments, indicating

reasonable approximation of DUE.

* The DTA procedure was computationally more intensive per iteration

than F-W but could handle dynamic scenarios that F-W could not.

- Limitations/Open Issues: - DTA is a heuristic and does not converge to an exact dynamic user-equilibrium solution.
- The link volume projection formula (Equation 16) is an approximation and may not perfectly predict future traffic.
- The assumption that link impedances are monotonically non-decreasing functions of flow is crucial.
- The paper notes that DTA is more computationally expensive per iteration than F-W, requiring more shortest path tree calculations.
- The DTA procedure requires an initial set of link loadings, which needs to be obtained by assigning trips for several time intervals prior to the analysis period.

- Relevance to my Research: - This paper's focus on dynamic traffic assignment and user equilibrium in urban networks is highly relevant to real-time urban navigation.
- The concept of projected link volumes to anticipate future congestion is a direct inspiration for how my Bidirectional A* can incorporate *predicted dynamic obstacle positions* and their impact on path costs.
- The formulation of link impedance as a function of volume (congestion) is a form of obstacle-aware penalty that can be adapted to my heuristic, where the cost of traversing a path segment increases with predicted obstacle density or risk.
- The DTA heuristic's ability to generate approximate solutions efficiently for large networks, even if not perfectly optimal, aligns with the need for real-time performance in urban navigation.
- The discussion of temporally continuous flows and the challenges of ensuring them in dynamic assignment highlights the importance of robust path validation in my research.

- Paper File Name : liu2010

- Paper Name: nan

- Citation: Liu, P., & Kim, I.-M. (2010). Performance Analysis of Bidirectional

Communication Protocols Based on Decode-and-Forward Relaying. IEEE

Transactions on Communications, 58(9), 2683-2696.

- Problem/Gap: - Unidirectional cooperative communication techniques suffer from low spectral efficiency due to the half-duplex constraint, especially in bidirectional information exchange scenarios.
- Straightforward application of unidirectional relaying to bidirectional networks also results in very low spectral efficiency.
- Lack of analytical results on outage probability and diversity-multiplexing tradeoff (DMT) for DF-based bidirectional protocols (TDBC, PNC, OSS).

- Algorithm/Method: - Three Decode-and-Forward (DF) Based Bidirectional Protocols Analyzed:
  - Time-Division Broadcast (TDBC): Combines transmissions of third and fourth time slots into a single broadcast, achieving one information exchange in three time slots. Achieves diversity order two.
  - Physical-Layer Network Coding (PNC): End-sources transmit simultaneously over a multiple-access channel, requiring only two time slots for one information exchange. Achieves diversity order one.
  - Opportunistic Source Selection (OSS): One end-source is opportunistically selected based on instantaneous channel conditions to maximize network throughput. Achieves diversity order two.
- Exact Closed-Form Outage Probabilities: Derived for PNC and OSS protocols, and an exact one-integral form for TDBC.
- Asymptotic Outage Probability Expressions: Derived for all protocols to analyze high-SNR performance.
- Diversity-Multiplexing Tradeoff (DMT) Analysis: Studied for all protocols in both finite and infinite SNR regimes to compare reliability and spectral efficiency.
- Asymptotic Optimal Power Allocation: Determined for each protocol.
- Asymptotic Optimal Relay Location: Determined for each protocol (found to be in the middle, d=0.5).

- Heuristic/Obstacle Handling: The paper focuses on optimizing communication protocols in wireless

networks, not pathfinding in physical space. Therefore, there are no direct "obstacle-aware penalties" in the context of urban

navigation.

* However, the concept of maximizing "diversity gain" (reliability)

and "multiplexing gain" (spectral efficiency) can be seen as

analogous to balancing path length/time (efficiency) with

safety/obstacle avoidance (reliability) in pathfinding.

* Opportunistic Source Selection (OSS): This protocol uses a heuristic of selecting the "best" channel condition at any given time, which

is a form of dynamic adaptation to "channel obstacles" (fading,

interference). This is conceptually similar to choosing the "best"

path segment based on real-time obstacle information.

- Key Results : - OSS protocol is more reliable than PNC and TDBC for low data rates (e.g., R = 1 bps/Hz) due to exploiting multiuser diversity.
- PNC protocol achieves the highest multiplexing gain, making it best for high data rates.
- TDBC and OSS protocols achieve higher maximal diversity gain (diversity order two) than PNC (diversity order one).
- Optimal relay location for all three protocols is in the middle (d=0.5) when SNR is sufficiently high.
- OSS protocol shows a performance gain of approximately 1.5 dB over TDBC in the high-SNR regime for certain power allocation.
- The optimal power ratio (β_o) is 2/3 for PNC, 1 for OSS, and between 2/3 and 1 for TDBC.

- Limitations/Open Issues: - The analysis assumes a straight-line model for relay location, which
is a simplification.
- The TDBC protocol's exact outage probability expression is in a complex one-integral form, making closed-form solution difficult.
- The paper focuses on DF-based relaying; AF-based relaying might have

different performance characteristics.
- The study uses a full Channel State Information (CSI) assumption, which may not always be practical in real-world scenarios.

- Relevance to my Research: - While not directly about pathfinding, this paper's rigorous analysis
of trade-offs between efficiency (multiplexing gain) and reliability (diversity gain) in communication protocols provides a valuable conceptual framework for my research.
- My Bidirectional A* algorithm needs to balance path efficiency (shortest time/distance) with safety (obstacle avoidance). The idea of optimizing for these competing objectives, as done with DMT, can inform my heuristic design.
- The concept of "opportunistic selection" based on current conditions (like OSS choosing the best channel) can be adapted to my algorithm to dynamically choose path segments or search directions based on real-time obstacle information and predicted risks.
- The analytical methods for deriving performance metrics (outage probability, DMT) can inspire how I quantitatively evaluate the performance of my Bidirectional A* with obstacle-aware penalties, especially in terms of path safety and efficiency.

- Paper File Name : jerbi2006

- Paper Name: nan

- Citation: Jerbi, M., Meraihi, R., Senouci, S.-M., & Ghamri-Doudane, Y. (2006). GyTAR: improved Greedy Traffic Aware Routing Protocol for Vehicular Ad Hoc Networks in City Environments. In VANET'06 (pp. 88-91). ACM.

- Problem/Gap: - Traditional Mobile Ad hoc NETworks (MANET) routing protocols fail to address the specific needs of Vehicular Ad Hoc NETworks (VANETs), especially in city environments (e.g., rapid topology changes, fragmented networks, signal transmissions blocked by obstacles, constrained mobility patterns).
- Existing routing protocols like Geographic Source Routing (GSR) and A-STAR often rely on simple greedy forwarding (closest vehicle to destination) and do not consider real-time traffic conditions, vehicle direction, or velocity.

- Algorithm/Method: - GyTAR (Improved Greedy Traffic Aware Routing Protocol): A new intersection-based geographical routing protocol designed for VANETs in urban environments.

- Two Modules:
  1. Junctions Selection: Dynamically chooses destination junctions based on a "score" that considers traffic density (Tj) between current and candidate junctions, and curvemetric distance (Dj) to the final destination. Score: $S(j) = \alpha \times f(Tj) + \beta \times g(Dj)$.
  2. Greedy Forwarding: Once a destination junction is determined, an improved greedy strategy is used. Each vehicle maintains a neighbor table with position, velocity, and direction, updating it periodically. The next hop neighbor is selected as the one closest to the destination junction.
- Recovery Strategy: Employs a "carry and forward" mechanism to prevent packets from getting stuck in local optima (e.g., no neighbor closer to destination).

- Heuristic/Obstacle Handling: - Traffic Density (`Tj`): This is a direct "obstacle-aware penalty." High traffic density on a road segment increases its cost (lowers its score), making it less likely to be chosen as part of the route.
- Curvermetric Distance (`Dj`): Geometric distance along the road network.
- Obstacles in City Environment: The protocol implicitly handles physical obstacles (buildings, etc.) by considering fragmented network conditions and signal blocking.
- Vehicle Speed and Direction: Incorporated into the forwarding decision, reflecting real-time traffic awareness.

- Key Results : - Simulation results (using Qualnet simulator) demonstrate that GyTAR achieves a higher packet delivery ratio compared to B-GyTAR (basic GyTAR without local recovery), DSR (Dynamic Source Routing), and LAR (Location Aided Routing).
- GyTAR shows lower control overhead compared to DSR and LAR, as it primarily uses hello messages for control and determines paths progressively based on road traffic density.
- The protocol effectively utilizes real-time traffic density information and movement prediction to find routes in urban environments.

- Limitations/Open Issues: - The evaluation is based on packet delivery ratio and control overhead, not directly on path length or travel time, which are key for real-time urban navigation.
- The "future work" mentions studying approaches for real-time inference of road densities, indicating that ground truth for

traffic density might be assumed or based on simpler models in this paper.
- While it addresses city environments, the specifics of how variable lane capacities, turns, or other complex urban features are fully integrated into Tj remain somewhat high-level.

- Relevance to my Research: - Real-Time Obstacle Proxy: This paper offers a strong precedent
for using "traffic density" as a proxy for "obstacle-awareness" in an urban setting. My "obstacle-aware penalties" can be directly mapped to traffic density or similar real-time metrics.
- Dynamic Cost Function: The score function S(j) demonstrates how to dynamically combine factors like "traffic density" and "distance" into a single cost metric. This is a direct parallel to how I can integrate obstacle-aware penalties into the cost function of Bidirectional A*.
- Urban Context Validation: The focus on city environments and VANETs validates the relevance of dynamic routing in scenarios pertinent to my research.
- Beyond Pure Distance: The paper moves beyond merely shortest path by considering traffic conditions, reinforcing the idea that "optimal route" is not just about physical distance but also about dynamic factors that introduce "penalties."
- Potential Integration: The greedy forwarding mechanism, while not A*, shows how local decisions can be made using combined dynamic information. This can inform how local node expansions in Bidirectional A* might utilize the obstacle-aware
penalties.

- Paper File Name : nannicini2011.pdf

- Paper Name: nan

- Citation: Nannicini, G., Delling, D., Schultes, D., & Liberti, L. (2012).

Bidirectional A* Search on Time-Dependent Road Networks.

Networks, *59*(2), 240-251.

- Problem/Gap: - Computing point-to-point shortest paths on time-dependent road networks is of large practical interest, but few efficient

algorithms exist.
- A major complication in time-dependent graphs is the difficulty of using bidirectional search, because the exact arrival time at the destination is unknown, making a standard backward search from the destination impossible.

- Algorithm/Method: - Time-Dependent Bidirectional A* (TDALT): A novel bidirectional search algorithm for time-dependent networks based on A* with landmarks (ALT).
- Backward Search with Lower Bounds: The key idea is to start a backward search from the destination node using time-independent lower bounds on arc costs. This backward search is not meant to find a path but to compute potential functions (estimates) for the forward search.
- Forward Search Pruning: The main forward search, which uses the actual time-dependent costs, is restricted by the bounds computed in the backward search. This prunes the search space, avoiding exploration of nodes that are unlikely to be on the shortest path.
- Three-Phase Approach:
  1. A bidirectional A* search runs on a time-independent graph (using lower-bound costs) until the search scopes meet.
  2. Both searches continue until the backward search queue only contains nodes whose keys exceed the current best solution cost (μ). This phase establishes a good upper bound and prunes the backward search.
  3. Only the time-dependent forward search continues, restricted to the set of nodes (M) settled by the backward search in the first two phases.

- Heuristic/Obstacle Handling: - Time-Dependency as an "Obstacle": The time-varying nature of arc
costs (e.g., rush hour traffic) is the primary "obstacle" that this algorithm is designed to handle efficiently.
- Landmarks (ALT): The A* heuristic is based on the ALT (A*, Landmarks, Triangle inequality) technique. Precomputed distances to a set of "landmark" nodes are used to provide a high-quality potential function (heuristic estimate) for the A* search, which is more effective than simple Euclidean distance.
- Lower-Bound Backward Search: The backward search uses a time-independent lower bound on arc travel times (e.g., length / max speed). This provides a "potential function" that guides and prunes the main time-dependent forward search.
- Approximation Factor (K): The algorithm can be run in an approximate mode by introducing a factor K. This allows for finding slightly

suboptimal paths much faster by pruning the search more
aggressively.

- Key Results : The bidirectional approach (TDALT) is significantly more effective

than unidirectional time-dependent ALT.

* With a small approximation factor (e.g., K=1.15), the algorithm

achieves a speedup of more than one order of magnitude compared to a
standard time-dependent Dijkstra's algorithm.

* On the European road network, with K=1.15, TDALT found solutions

with an average relative error of only 0.3% while being over 7 times
faster than unidirectional ALT and 26 times faster than Dijkstra.

* The performance gain is especially significant for long-distance

queries.

* The use of "tightened bounds" (an improved potential function for

the backward search) provides a great deal of the computational

improvement.

- Limitations/Open Issues: - The algorithm's performance depends on the quality of the landmarks
and the lower bounds.
- The method requires the graph to satisfy the FIFO (First-In,
First-Out) or non-overtaking property.
- For very short-range queries, the overhead of the bidirectional
approach can make it slightly slower than a unidirectional one.
- The exact version of the algorithm (K=1.0) is slower than
unidirectional ALT, so a willingness to accept small approximations
is key to its effectiveness.

- Relevance to my Research: - This paper is highly relevant as it directly addresses bidirectional
search on time-dependent networks, which is a core component of my
proposed research.
- The central idea of using a time-independent backward search to
generate heuristics/bounds for a time-dependent forward search is a

powerful technique I can adapt. My research can use a similar approach where the backward search ignores dynamic obstacles (or uses a simplified, static representation of them) to create an effective heuristic for the forward search that is aware of them.
- The use of an approximation factor (K) to trade optimality for speed is a crucial concept for real-time applications. My algorithm could incorporate a similar parameter to control the trade-off between path optimality and computation time, which is essential for urban navigation.
- This paper validates that bidirectional search can be successfully adapted to time-dependent problems, providing a strong foundation and justification for my work on enhancing Bidirectional A* with obstacle-aware penalties.

- Paper File Name : sabar2019.pdf

- Paper Name: nan

- Citation: Sabar, N. R., Bhaskar, A., Chung, E., Turky, A., & Song, A. (2018). A self-adaptive evolutionary algorithm for dynamic

vehicle routing problems with traffic congestion. Swarm and Evolutionary Computation BASE DATA.

- Problem/Gap: - Addresses Dynamic Vehicle Routing Problems (DVRP) with varying travel times due to traffic congestion.
- Need for algorithms to adapt to dynamic changes and continuously find optimal solutions in real-time.
- Performance of Evolutionary Algorithms (EAs) is highly dependent on fixed configurations (parameters, operators), leading to sub-optimal results in dynamic problems.
- Existing adaptive EA methods often focus on a limited number of parameters/operators and do not consider complex configurations or the sequence of operator application.

- Algorithm/Method: - Self-adaptive Evolutionary Algorithm (SAEA): A proposed framework that dynamically adjusts GA configurations (parameters, operator types, combinations, and application sequences) along with the solutions.
- Integrated Genetic Algorithm (GA): SAEA is built upon a classical GA framework.
- Solution Representation: Each individual in SAEA has three parts:

  - Part 1 (DVRP Solution): Path representation with route delimiters (sequence of customer indices).
  - Part 2 (Parameters): Encodes numeric parameters like crossover rate (P1) and mutation rate (P2).
  - Part 3 (Operators): Encodes three types of operators:
    - OP1 (Crossover): Order-based, Route-based, and Swap-based crossover operators.
    - OP2 (Mutation): Random remove, Worst remove, and Reverse mutation operators.
    - OP3 (Improvement/Local Search): Swap, Single move, and Double move operators.
- Uses roulette wheel selection for reproduction.
- A decoder converts selected individuals into DVRP solutions and configurations, which are then applied.

- Heuristic/Obstacle Handling: Dynamic Traffic Factor (tij): The DVRP model incorporates a traffic factor tij that is added to all edges connecting

customers.

* Traffic Factor Generation: tij is generated dynamically

based on a magnitude of changes (mt) and a random number

(Rnd) within defined lower (FL) and upper (FU) bounds,

simulating varying congestion levels.

* Asymmetric Travel Times: Dynamic changes in traffic lead to asymmetric travel times between locations.

* The self-adaptive nature of SAEA itself acts as a

meta-heuristic to adapt to these dynamic traffic conditions by evolving optimal configurations.

- Key Results : - SAEA achieved new best results for all tested DVRP instances and 85.4% of DTSP instances.
- Statistically significant improvement over 11 other EA variations (EA1-EA11) across all tested instances in terms of best, average, and standard deviation of objective function values.
- Outperformed state-of-the-art algorithms for DVRP (e.g., ASrank-CVRP, ACS-DVRP, M-ACO) and DTSP (e.g., MMAS, KP,

EIACO) on most instances.
- Demonstrated superior ability to cope with dynamic changes and find high-quality solutions.

- Limitations/Open Issues: - Future work includes applying the proposed algorithm to real-world dynamic vehicle routing problems.
- The paper does not explicitly detail limitations of the SAEA itself, but the complexity of evolving configurations could be a practical consideration.

- Relevance to my Research: - Dynamic Adaptation: The core concept of a self-adaptive algorithm that evolves its own parameters and operators in dynamic environments is highly relevant for enhancing Bidirectional A* in real-time urban navigation.
- Obstacle-Aware Penalties: The explicit modeling of traffic congestion as a dynamic factor affecting travel times (via tij) provides a direct example and methodology for incorporating "obstacle-aware penalties" into my Bidirectional A* approach.
- Problem Alignment: The paper's focus on DVRP with traffic congestion directly aligns with the challenges of real-time urban navigation, where dynamic obstacles and traffic are prevalent.
- Performance Validation: The strong performance of SAEA in dynamic settings reinforces the importance of adaptive mechanisms for pathfinding algorithms in such environments.
- Inspiration for Heuristics: The dynamic generation of traffic factors and the self-adaptive nature of the algorithm can inspire new ways to design adaptive heuristics or penalty functions for Bidirectional A*.

- Paper File Name : silver2004.pdf

- Paper Name: nan

- Citation: Silver, E. A. (2004). An overview of heuristic solution

methods. Journal of the Operational Research Society,

55(9), 936–956.

- Problem/Gap: - Addresses the need for usable solutions to well-defined mathematical models of real-world problems where finding the mathematically optimal solution is difficult or

impossible.
- Highlights that achieving optimality often requires oversimplifying the model, making it less representative of the real problem. It's better to have a reasonable solution to an accurate model than an optimal solution to an inaccurate one.
- Complexity arises from combinatorial explosion, stochastic variables making the objective function hard to evaluate, and time-varying conditions.

- Algorithm/Method: - This paper is a survey and does not propose a single new algorithm. It categorizes and describes a wide range of heuristic and metaheuristic methods.
- Basic Heuristic Types:
  - Problem Decomposition/Partitioning: Break a complex problem into simpler subproblems (e.g., rolling horizon).
  - Methods that Reduce Solution Space: Cut back on the number of solutions considered (e.g., beam search, feature extraction).
  - Approximation Methods: Manipulate the mathematical model itself (e.g., aggregate parameters, modify the objective function, relax constraints).
  - Constructive Methods: Build a solution step-by-step (e.g., greedy algorithms like nearest neighbor for TSP).
  - Local Improvement (Neighborhood Search): Start with a feasible solution and iteratively move to better solutions in its "neighborhood" until a local optimum is found.
- Metaheuristics (to escape local optima):

  * Tabu Search: A local search that permits moves to

  inferior solutions but uses a "tabu list" of recent

  solutions to avoid cycling.

  * Simulated Annealing: A probabilistic local search that

  accepts inferior moves with a probability that

  decreases over time (controlled by a "temperature"

parameter).

* Variable Neighbourhood Search: Uses a set of nested

neighborhood structures to systematically explore

different areas of the solution space.

* Guided Local Search: Adds a weighted set of penalty

functions to the objective function to guide the search
away from previously found local optima.

* Ant Colony Search: Emulates ant behavior, where paths

are probabilistically chosen based on distance and the

amount of "pheromone" left by previous "ants."

* Evolutionary Algorithms (e.g., Genetic Algorithms):

Works with a population of solutions, using operators

like crossover (mating) and mutation to evolve better

solutions over generations.

- Heuristic/Obstacle Handling: - The paper discusses handling complexity in general terms rather than specific obstacle-handling techniques.
- Approximation Methods are the most relevant category. This includes:
  - Approximating stochastic processes: Assuming random variables are constant at their mean values or using analytically convenient distributions.
  - Changing the nature of constraints: Relaxing constraints (e.g., Lagrangian relaxation) to make the problem easier to solve, then adjusting the solution to regain feasibility. This is a way to "handle" the "obstacle" of a difficult constraint.
- The core idea is to simplify the problem representation to make it solvable, which is analogous to abstracting or simplifying obstacles.

- Key Results : This is a survey paper, so it reports on the state of the

field rather than presenting new quantitative results.

* It emphasizes that heuristics are widely used and effective
in practice for a vast range of complex combinatorial

problems (e.g., TSP, vehicle routing, scheduling).

* It provides a framework for classifying and understanding

the large landscape of heuristic and metaheuristic

techniques.

- Limitations/Open Issues: - The paper acknowledges that the choice and tuning of a heuristic for a specific problem is a creative undertaking and depends on many factors (problem size, time available, etc.).
- A key challenge is performance evaluation: comparing a heuristic solution to the unknown optimal solution. The paper discusses methods like comparison with bounds (from relaxation), worst-case analysis, and experimental analysis on test problem sets.

- Relevance to my Research: - Methodology Framework: Provides a comprehensive classification of heuristic and metaheuristic approaches. This is invaluable for situating my own work (enhancing Bidirectional A*) within the broader field of heuristic optimization and for my literature review.
- Inspiration for Hybrid Approaches: The discussion of hybrid methods (e.g., combining a population-based method with a local search) is highly relevant. I could consider hybridizing my Bidirectional A* with other techniques described, such as a local search to refine paths or concepts from Guided Local Search to penalize congested areas.
- Understanding Search Strategies: The concepts of "intensification" (focused local search) and "diversification" (broader exploration to escape local optima) are fundamental. My "obstacle-aware penalties" can be seen as a form of guided search that discourages movement into certain regions, relating to the principles of Tabu Search or Guided Local Search.

- Evaluation Techniques: The section on performance evaluation offers a structured way to think about how to test and validate my proposed algorithm, for instance, by comparing its results against bounds or other established heuristics on benchmark urban navigation problems.

- Paper File Name : shabani2019

- Paper Name: nan

- Citation: * Shabani, A., Asgarian, B., Gharebaghi, S. A., Salido, M. A., &

- Problem/Gap: - Many real-world optimization problems are complex and have many local optima, making it difficult for traditional algorithms to find the global optimum.
- There is a need for efficient optimization methods for real-world engineering design problems.
- While many metaheuristic algorithms are inspired by nature (e.g., swarm intelligence, physics), few have been based on human search behaviors, specifically those in search and rescue operations.

- Algorithm/Method: - Search and Rescue Optimization Algorithm (SAR): A new human-based metaheuristic algorithm for solving single-objective continuous optimization problems.
- Inspiration: The algorithm is inspired by the explorations carried out by humans during search and rescue (SAR) operations.
- Two-Phase Search: The algorithm models the two main phases of human search in SAR operations:
  1. Social Phase: Group members (humans) search based on the position of found "clues" and their quality. The search direction is determined by the vector from the human's current position to a randomly selected clue.
  2. Individual Phase: Humans search around their current position, independent of other clues, to explore the immediate vicinity.
- Memory and Clues: The algorithm maintains a "clues matrix" which stores the positions of all found clues (both "held" and "abandoned"). This matrix is used to generate new potential solutions in both phases. A memory matrix stores past positions to increase diversity.

- Heuristic/Obstacle Handling: - Clue-Based Search: The "clues" (good solutions found so far) act as attractors, guiding the search. The algorithm uses the objective

function value of clues to decide the search direction, prioritizing better clues. This is a heuristic for navigating the solution space.
- Social vs. Individual Phase: The balance between the social phase (exploration, guided by clues from others) and the individual phase (exploitation, searching locally) is a core heuristic of the algorithm.
- Abandoning Clues: A mechanism to prevent getting stuck in local optima. If a "human" (search agent) fails to find a better clue after a certain number of attempts (Maximum Unsuccessful Search - MU), they abandon the current position and jump to a new random position in the search space.
- Boundary Control: Ensures that all generated solutions remain within the feasible search space.

- Key Results : SAR was evaluated on 55 optimization functions (27 classic, 28

modern CEC 2013) and compared against twelve other optimization

algorithms.

* Statistical results (Wilcoxon signed-rank test) indicated that SAR

is highly competitive and often superior to well-known algorithms

like GA, DE, PSO, and recent metaheuristics.

* On three real-world engineering design problems (I-Beam, Cantilever

Beam, 25-Bar Truss), SAR was able to find more accurate solutions

with fewer function evaluations compared to other existing

algorithms.

* SAR demonstrated a strong global search ability and a fast

convergence rate.

- Limitations/Open Issues: - The paper focuses on single-objective continuous optimization problems. The performance on other types, like combinatorial and large-scale optimization problems, is identified as an area for future work.
- The control parameters (SE - social effect, MU - maximum

unsuccessful searches) were set based on analysis, but their optimal values might be problem-dependent.

- Relevance to my Research: - This paper provides a novel, human-inspired metaheuristic that balances exploration and exploitation, which is a key challenge in pathfinding in dynamic environments.
- The two-phase search (social and individual) is analogous to a pathfinding strategy that might combine global information (e.g., from a coarse map or other agents) with local, real-time sensor data. My Bidirectional A* could be viewed as a form of social/cooperative search.
- The concept of "clues" as attractors in the search space is directly transferable to my research. Dynamic obstacles or high-penalty areas could be treated as negative "clues" or repellents, pushing the search away from them.
- The "abandoning clues" mechanism to escape local optima is a valuable heuristic. My algorithm could incorporate a similar strategy: if a search direction consistently leads to high-cost or blocked paths, it could be temporarily abandoned or heavily penalized to encourage exploration of other routes.

- Paper File Name : wan2019

- Paper Name: nan

- Citation: Wan, X., Ghazzai, H., & Massoud, Y. (2019). Real-Time

Navigation in Urban Areas Using Mobile Crowd-Sourced Data.

2019 IEEE International Smart Cities Conference (ISC2).

- Problem/Gap: - Traditional navigation solutions (e.g., Google Maps, Waze) rely on historical data and statistical records, which are not always effective in complex urban environments with frequent, unexpected events like constructions, accidents, and blocked roads.
- Existing methods may guide many drivers to the same "optimal" route, causing it to become congested, thus unbalancing traffic flow.
- There is a lack of navigation systems that can instantly guide a vehicle based on real-time feedback from a variety

of road network users (other vehicles, sensors, pedestrians).

- Algorithm/Method: - Integer Linear Program (ILP): The core of the method is an ILP formulation to determine the optimal route.
- System Model: The urban area is modeled as a network of roads and intersections. Roads are divided into smaller segments.
- Real-Time Data Inputs: The system uses a central cloud server to continuously collect real-time, crowd-sourced data:
  - Vi,j,t: Traffic speed on a segment.
  - Ti,j,t: Expected waiting time at a segment (e.g., for a traffic light).
  - ei,j,t: Road status (binary: blocked or not).
- Utility Function: A weighted utility function Us(xi,j,s) = w*fs + (1 − w)*gs is optimized at each step s.
  - fs: Fitness function representing the expected time spent on each segment (on-line, real-time data).
  - gs: Distance function representing the geographical distance to the destination (off-line data).
  - w: A Pareto parameter to balance the trade-off between the fastest route (on-line) and the shortest path (off-line).
- Two Iterative Algorithms:
  1. Window-Size Algorithm (WSA): Re-calculates the entire route from the vehicle's current position to the destination every Ws seconds, based on the latest data. This is computationally expensive.
  2. Low Complexity Window Size Algorithm (LCWSA): To reduce complexity, this algorithm re-calculates the route for only a limited region (a circular area with radius RE) around the vehicle's current location every Ws seconds.

- Heuristic/Obstacle Handling: Explicit Obstacle Modeling: Obstacles are handled directly

through the real-time data input ei,j,t, which is a binary

variable indicating if a road segment is blocked.

* Penalty Factor: A large penalty factor Eo is added to the

fitness function fi,j,t for any segment (i, j) that is

reported as blocked (ei,j,t = 1), effectively making that

path prohibitively "costly" and ensuring the algorithm

avoids it.

* Congestion as a "Soft" Obstacle: Traffic congestion is

handled by incorporating real-time speed Vi,j,t and waiting
times Ti,j,t into the fitness function. Slower speeds and

longer wait times increase the cost of a segment, naturally
guiding the algorithm away from congested areas.

- Key Results : - Both WSA and LCWSA consistently outperform the standard
Shortest Path Algorithm (SPA), especially when roads are
unexpectedly blocked.
- WSA and LCWSA saved over 30% of travel time compared to SPA
when a key road was blocked.
- WSA consistently performed the best, saving over 20% travel
time even with a blocked road and 14% without, efficiently
exploiting real-time feedback.
- LCWSA, while less optimal than WSA, still significantly
outperformed SPA and offers a good trade-off between
performance and computational complexity.
- The choice of the window size Ws is critical; a value
around 120 seconds was found to provide the best results in
their empirical tests, balancing data freshness with
computational overhead.

- Limitations/Open Issues: - The proposed methods assume the real-time crowd-sourced
data is perfect and trustworthy. The paper acknowledges
that anomalies, errors, or absence of reported data need to
be addressed in future work.
- The performance is sensitive to the choice of the window
size Ws. A poor choice can lead to degraded performance.
- The paper does not deeply explore the design of the data
processing/filtering function F(fi,j,t) at the server,
which is critical for handling noisy or conflicting
real-world data.

- Relevance to my Research: Directly Applicable Model: The paper's core idea of using
an ILP with a utility function that blends real-time

(on-line) data with static (off-line) map data is directly relevant to my goal of enhancing Bidirectional A*.
* Obstacle-Aware Penalty Implementation: The use of a large penalty factor Eo for blocked roads is a concrete, simple, and effective example of an "obstacle-aware penalty" that I can adapt for my algorithm.
* Dynamic Updates: The iterative, window-based approach (WSA/LCWSA) provides a practical framework for how a real-time navigation system can periodically re-plan paths, which is a key requirement for my research.
* Contrast with A*: This paper uses an ILP solver, which is different from my A*-based approach. This provides a valuable point of contrast. I can argue that a heuristic search like Bidirectional A* could be more computationally efficient for very large-scale networks compared to solving an ILP, especially if my obstacle-aware penalties are well-designed.
* Parameterization: The use of the Pareto weight w to balance on-line vs. off-line factors is an insightful technique that I could incorporate into my heuristic function for Bidirectional A*.

- Paper File Name : sud2008

- Paper Name: nan

- Citation: Sud, A., Gayle, R., Andersen, E., Guy, S., Lin, M., & Manocha, D.

(2008). Real-time Navigation of Independent Agents Using Adaptive

Roadmaps. In Proceedings of the 2008 ACM SIGGRAPH/Eurographics

Symposium on Computer Animation (SCA '08) (pp. 1-10).

- Problem/Gap: - Navigating a large number of independent agents in complex, dynamic environments (e.g., virtual reality, crowd simulation) in real-time is challenging.
- Existing global path planning algorithms are often too slow for real-time multi-agent scenarios or are limited to static environments.
- Local collision avoidance methods suffer from local minima problems and lack guarantees for finding collision-free paths.

- Traditional roadmap-based algorithms do not scale well to environments with many independent agents or dynamic obstacles.

- Algorithm/Method: - Adaptive Elastic Roadmaps (AERO): A novel algorithm for real-time navigation of large-scale heterogeneous agents in complex dynamic environments.
- Global Connectivity Graph: AERO uses a global roadmap (a graph of milestones and links) that continuously deforms based on obstacle motion and inter-agent interaction forces.
- Physically-Based Particle Simulator: Used to compute and update AERO. Milestones and links are represented as particles connected by springs, which respond to internal (maintaining link length) and external (repulsive from obstacles) forces.
- Link Bands: Introduced to augment local dynamics and resolve collisions among multiple agents. A link band is the region of free space closer to a specific link than any other.
- Lazy and Incremental Updates: The roadmap is updated lazily and incrementally, meaning updates only occur when necessary (e.g., when links are deformed beyond a threshold or obstacles move into a link band).
- A* Graph Search: Used to compute paths on the weighted roadmap.
- Link Removal/Addition: Links are removed if they are too deformed (high potential energy) or too close to obstacles. New links are added to maintain connectivity, prioritizing previously removed links.

- Heuristic/Obstacle Handling: Dynamic Obstacles and Inter-Agent Forces: Moving obstacles and other
agents are treated as dynamic elements that exert repulsive forces

on the roadmap's particles, causing the roadmap to deform and avoid

collisions.

* Link Bands as Collision Zones: Link bands define collision-free

zones around roadmap links. Agents are guided along these bands, and crossing link band boundaries triggers re-planning events.

* Cost Function for A*: The A* algorithm uses a cost function that

combines link length, the reciprocal of link band width (penalizing

narrow passages), and agent density on the link (penalizing crowded

regions).

* Penalty for Crowded Regions: A higher value for the agent density

term in the cost function causes agents to plan using less crowded

regions, effectively treating crowds as "soft obstacles."

* Repulsive Forces: Obstacles and other agents exert repulsive forces

on the roadmap, pushing the path away from them. This is a direct

obstacle-aware penalty.

- Key Results : - AERO can perform real-time navigation for hundreds and thousands of independent agents in complex indoor and outdoor scenes (e.g., 1,000 agents at 16fps in a city scene).
- The algorithm successfully handles dynamic obstacles and inter-agent interactions, continuously adapting the roadmap.
- The use of link bands and efficient force computation allows for fast local collision avoidance.
- Demonstrated on complex scenarios like a maze, a tradeshow, and a city scene, showing emergent crowd behaviors (e.g., lane formation, re-routing around cars).
- The approach provides global path planning guarantees, unlike purely local methods.

- Limitations/Open Issues: - The current implementation addresses 3-DoF (degrees of freedom) agents, which may not produce realistic motion for high-DoF avatars.
- AERO, while global, can still get agents stuck in local minima across space-time, meaning it doesn't guarantee convergence or completeness for collision-free paths in all environments.
- The performance of proximity queries is sensitive to the choice of hash function parameters.
- The approach does not fully exploit all behavior-related characteristics of real crowds, such as grouping.

- Relevance to my Research: This paper's Adaptive Elastic Roadmaps (AERO) provide a highly relevant framework for incorporating dynamic obstacle avoidance into pathfinding for real-time urban navigation. * The concept of a dynamically deforming roadmap, influenced by repulsive forces from obstacles and other agents, is a direct inspiration for how to implement obstacle-aware penalties in my Bidirectional A\. The "elasticity" of the

roadmap is analogous to how my heuristic would dynamically adjust costs based on obstacle proximity and movement. * The use of link bands to define collision-free zones and trigger re-planning events is a practical mechanism for handling dynamic environments. This could be adapted to define "safe corridors" in my urban navigation system. * The A\ cost function that penalizes narrow passages and crowded regions is a direct example of how to build obstacle-aware penalties into the heuristic, not just for static obstacles but also for dynamic congestion. * The lazy and incremental update strategy is crucial for real-time performance, a principle I must adopt for my Bidirectional A* to be effective in urban settings.

- Paper File Name : yan2018

- Paper Name: nan

- Citation: Yan, L., Hu, W., & Hu, S. (2018). SALA: A Self-Adaptive

Learning Algorithm—Towards Efficient Dynamic Route Guidance
in Urban Traffic Networks. Neural Process Lett, 48(1),

291–309. https://doi.org/10.1007/s11063-018-9870-0

- Problem/Gap: - Existing traffic management methods (signal optimization, traffic assignment) often fail to provide fast, accurate, and personalized route guidance due to a lack of individual traffic demands, real-time data, and dynamic cooperation between vehicles.
- Traditional approaches struggle with the scalability of transmitting traffic information and do not adequately account for unexpected incidents (e.g., accidents).
- The challenge is to achieve user-optimal routes while also considering the overall system efficiency and avoiding congestion caused by all vehicles choosing the same "best" route.

- Algorithm/Method: - Dynamic and Real-time Route Selection Model (DR2SM): A multi-agent system where individual vehicle agents simulate driving vehicles in an urban traffic network.
- Self-Adaptive Learning Algorithm (SALA): Each vehicle agent uses SALA to play a stochastic congestion game, learning from historical experiences and observations of other vehicles to reach a mixed Nash equilibrium.
- Route Selection Process: Each vehicle selects a user-optimal route that maximizes its utility.

- Utility Calculation: The utility of a route for a vehicle is based on three factors:
  - Preference Value: Driver's preferences (e.g., route familiarity, road conditions, light conditions, compliance with guidance, sidewalk presence).
  - Uncertainty Value: Accounts for unexpected incidents like accidents or temporary activities, with values ranging from 0 to INF (indicating a blocked road).
  - Cost Value: Estimated time cost, distance cost, and oil consumption cost, influenced by congestion degree.
- Communication: Uses both Internet (for centralized Traffic Information Provider - TIP) and VANETs (Vehicular Ad-hoc Networks) for local vehicle-to-vehicle negotiation.
- Scalability: Offloads route computation to individual vehicles, with TIP only updating graph attributes, reducing CPU load and network traffic.

- Heuristic/Obstacle Handling: Uncertainty Value for Incidents: Explicitly models

unexpected incidents (accidents, activities) as an

"uncertainty value" that can range to INF, effectively

penalizing or blocking routes.

* Congestion Game: Vehicles play a "congestion game" to

negotiate routes, implicitly handling the "obstacle" of

other vehicles' presence and potential congestion.

* Cost Factors: Time, distance, and oil consumption costs are dynamically calculated based on congestion degree, acting

as penalties for less efficient or more congested paths.

* Self-Adaptive Learning: The SALA itself is a heuristic

approach that allows vehicles to adapt their route choices

based on real-time conditions and the behavior of other

agents, effectively navigating dynamic obstacles (traffic,

incidents).

- Key Results : - DR2SM effectively reduces average traveling time in dynamic and uncertain urban traffic networks.
- Compared to non-cooperative route selection algorithms (SPA, PA, PSPA) and state-of-the-art equilibrium algorithms (WEA, RSGA, CARAVAN), DR2SM significantly reduces average travel time (10-36% decrease over non-cooperative, 8-22% over equilibrium algorithms).
- DR2SM achieves the highest percentage of decrease in travel time when road saturation is around 0.5.
- DR2SM has the least data communication cost and lowest stop number compared to other equilibrium algorithms.
- The algorithm shows better performance with a larger number of intersections, as it provides more opportunities for cooperation.

- Limitations/Open Issues: - Assumes all vehicles follow the route guidance system; real-world driver disobedience needs further study.
- Traffic accidents were quantified in the utility function but not explicitly simulated as dynamic events in the experiments. Future work will focus on more complex simulations of traffic accidents as triggers for rerouting.
- The time complexity of SALA is relatively high (O(nm) for dynamic programming), though it is argued that hardware advancements can mitigate this.

- Relevance to my Research: Multi-Agent Perspective: The multi-agent system and congestion game approach offer a novel perspective on dynamic pathfinding, which could inspire how Bidirectional A* could interact with other agents or dynamically adjust its search based on predicted collective behavior.
* Comprehensive Utility Function: The detailed utility function incorporating driver preferences, uncertainty (for incidents), and various costs (time, distance, oil) provides a rich model for developing sophisticated obstacle-aware penalties in my Bidirectional A* heuristic.
* Dynamic Obstacle Modeling: The explicit inclusion of "uncertainty value" for incidents (accidents, activities) and dynamic cost factors for congestion directly informs how I can design and integrate real-time, dynamic obstacle information into my pathfinding algorithm.
* Scalability Considerations: The strategy of offloading

computation to individual agents and using both centralized and local communication is a valuable lesson for designing a scalable Bidirectional A* for large urban networks.
* Nash Equilibrium: The concept of reaching a Nash equilibrium in a congestion game could be a theoretical underpinning for how my Bidirectional A* could find paths that are not just individually optimal but also contribute to overall system efficiency in a multi-agent urban environment.

- Paper File Name : song2018

- Paper Name: nan

- Citation: Song, A.L., Su, Y., Dong, Z., Shen, W., Xiang, Z., & Mao, P. (2018). A two-level dynamic obstacle avoidance algorithm for unmanned surface vehicles. Ocean Engineering, 170, 351–360.

- Problem/Gap: - **Emergency avoidance gap**: Little research addresses USV obstacle avoidance when obstacles violate COLREGS rules or exhibit unpredictable movement
- **Real-world constraint**: Existing algorithms fail when obstacles actively approach the USV or maintain irregular motion patterns
- **Motion capacity limitation**: Current methods don't adequately account for USV speed/steering constraints during collision avoidance
- **Overlapping stalemate**: Specific failure mode where USV cannot overtake due to similar speeds or obstacle's reactive turning

- Algorithm/Method: - **Two-level hierarchical approach**:
  - **Level 1 (Non-emergency)**: Velocity Obstacle (VO) algorithm combined with PSO optimization for COLREGS-compliant avoidance
  - **Level 2 (Emergency)**: Improved Artificial Potential Field (APF) with composite force field
- **Emergency triggers**:
  - General: USV enters obstacle domain ($A \cap D \neq \emptyset$)
  - Overlapping stalemate: Speed difference < 20%, course difference < 30°, persisting for 5+ cycles
- **PSO integration**: Optimizes velocity change (Δv_R) and course change ($\Delta\alpha$) to minimize motion variation while satisfying avoidance constraints

- Heuristic/Obstacle Handling: - **VO-based penalty**: Maintains USV outside cone-shaped velocity obstacle space; uses angle γ relative to tangent μ to determine collision risk
- **Composite potential field** (Emergency):
  - **Repulsive force**: $U_{Re} = \frac{1}{2}\eta(1/\rho - 1/\rho_0)^2$ (pushes away from obstacle)
  - **Centrifugal force**: F_Rot = F_Re rotated 90° toward obstacle stern (guides around

obstacle)
  - Weight-adjustable combination: $F_{all} = \omega_1 F_{Rot} + \omega_2 F_{Re}$
- **Motion constraints**: Speed limit $\bar{v}_R$, acceleration limit $\bar{a}$, course change limit $\bar{w}$ enforced in real-time
- **COLREGS integration**: Overtaking ($\Delta\theta < 45°$), head-on ($|180° - \Delta\theta| < 15°$), crossing ($45° \leq |\Delta\theta| \leq 165°$)

- Key Results : - **Single obstacle**: 99% collision avoidance success rate over 100 simulations (uniform, variable, rectilinear, curvilinear motion)
- **Multi-obstacle (2-5 boats)**: 95% success rate over 100 simulations
- **Computational efficiency**: 5-second update cycle using PSO for parameter optimization
- **Emergency response**: Successfully handled deliberate blocking scenarios, high-speed head-on encounters, and overlapping stalemate situations
- **5% failures**: Only in extreme cases where 4+ obstacles deliberately block with speeds exceeding USV capacity

- Limitations/Open Issues: - **Sandwiched scenarios**: No strategy for when USV is surrounded/encircled by multiple purposefully colliding obstacles
- **Observability assumption**: Assumes AIS-equipped obstacles with negligible observation error
- **Motion coupling ignored**: Simplifies by not modeling speed-heading coupling during steering
- **Static field assumptions**: Potential field parameters (η, ρ_0, ω_1, ω_2) appear fixed; no adaptive weighting mechanism
- **Non-cooperative obstacles**: 5% failure cases suggest fundamental limits when obstacles have superior dynamics and coordinated blocking
- **Computational scalability**: PSO convergence time not reported; unclear if real-time guarantees hold for dense traffic

- Relevance to my Research: - **Hierarchical switching concept**: Your Bidirectional A* enhancement could incorporate similar emergency-mode switching when obstacles cluster near planned path
- **Obstacle-aware penalties framework**: The composite potential field (repulsive + centrifugal) demonstrates **combining multiple penalty types** for different threat levels—directly applicable to defining obstacle penalties in A* cost function
- **Dynamic constraint handling**: Their PSO-enforced motion limits ($\bar{v}$, $\bar{a}$, $\bar{w}$) show importance of **kinematic feasibility** in penalty design—your A* should penalize paths requiring unrealistic maneuvers
- **Quantitative thresholds**: Specific emergency triggers (domain overlap, speed/course differences, time persistence) provide metrics for **when to apply higher obstacle penalties** in your heuristic
- **Contrast**: VO is reactive (local adjustments); your Bidirectional A* is planning-based (global path)—hybrid approach could use A* for strategic planning with VO-inspired penalties for dynamic re-weighting

- **Urban navigation parallel**: Emergency/non-emergency distinction maps to urban scenarios (e.g., pedestrian suddenly enters crosswalk vs. parked car)—tiered penalty system applicable
- **Performance baseline**: 95-99% success with 5s cycles sets benchmark; your real-time urban A* should target similar success rates with comparable/faster computation

- Paper File Name : Research on Robot Dynamic Obstacle Avoidance Method Based on Impr

- Paper Name: nan

- Citation: Zhang, Y., Li, B., Huo, T., & Liu, R. (2025). Research on Robot Dynamic Obstacle Avoidance Method Based on Improved A* and Dynamic Window Algorithm. Journal of System Simulation, 37(6), 1555-1564.

- Problem/Gap: - **Excessive expansion nodes**: Traditional A* generates too many nodes during path search
- **Redundant turning points**: Paths contain unnecessary directional changes
- **Static planning limitation**: A* cannot handle dynamic obstacles in complex environments
- **Four-neighborhood inefficiency**: Classical 4-neighborhood expansion produces redundant nodes
- **Eight-neighborhood safety issue**: 8-neighborhood expansion creates paths that cut through diagonal obstacles or pass through obstacle vertices
- **Collinear redundancy**: Planned paths contain unnecessary intermediate nodes on straight segments

- Algorithm/Method: - **Hybrid architecture**: Improved A* for global planning + DWA for local dynamic avoidance
- **Adaptive dual-neighborhood expansion**:
  - Diagonal-eight-neighborhood: Extends {(x±2,y), (x,y±2), (x±1,y±1)} for open areas
  - Four-neighborhood: Switches to {(x±1,y), (x,y±1)} when obstacles detected in cardinal directions or near goal
  - Adaptive selection based on obstacle presence in 4-directional neighbors
- **Quadrant selection method**: Directly calculates coordinate differences (Δx, Δy) to constrain search direction to target quadrant—eliminates angle computation (θ = arctan)
- **Redundant point elimination**: Iterative 3-node window checks if $node_1$-$node_3$ line segment intersects obstacles; removes $node_2$ if clear
- **DWA integration**: Uses A* waypoints as intermediate goals for DWA local planning between consecutive nodes

- Heuristic/Obstacle Handling:

- **Cost function**: f(n) = g(n) + h(n), where h(n) uses Manhattan distance
- **Obstacle-aware switching**: Checks 4-neighbor positions for obstacles before selecting

expansion strategy
- **Quadrant-based directional bias**: Prioritizes 3 expansion nodes in target quadrant; shifts to adjacent quadrants if all 3 blocked
- **Safety verification**: Line-of-sight check between non-adjacent nodes prevents shortcut through obstacles during redundancy removal
- **DWA evaluation function**: $G(v,\omega) = \alpha H(v,\omega) + \beta G(v,\omega) + \gamma P(v,\omega) + \sigma O(v,\omega)$
  - H: Heading deviation from goal
  - G: Distance to goal
  - P: Distance from trajectory endpoint to global path (keeps robot on A* path)
  - O: Obstacle clearance

- Key Results : - **Environment I (10×10, 14% obstacles)**:
  - Algorithm IV (full method) vs Traditional A*: 11.9% longer path but 52% fewer expansion nodes, 50% fewer turns
  - Eliminated unsafe passages through obstacle gaps present in traditional A*
- **Environment II (20×20, 24% obstacles)**:
  - 70% reduction in expansion nodes (40→12), 43% reduction in path turns (11→13 intermediate, final 13)
  - 6.4% faster runtime (9.368s→8.759s) despite path optimization overhead
- **Comparison benchmarks (20×20 maps)**:
  - vs Dijkstra: Slightly longer path (+8.2%) but 19.4% faster, 0 dangerous nodes (Dijkstra: 9)
  - vs Angle search: 6.0% longer but 0 dangerous nodes (Angle search: 9), 50% fewer turns
- **Dynamic avoidance**: Successfully avoided 4 moving obstacles and random obstacles, maintaining global optimality

- Limitations/Open Issues: - **Grid resolution dependency**: Performance metrics tied to specific grid sizes (10×10, 20×20); scalability to larger urban maps unclear
- **DWA computational cost**: Runtime for DWA between each A* waypoint not separately reported—potential real-time bottleneck in dense obstacle fields
- **Quadrant switching logic**: When all 3 target-quadrant nodes blocked, criterion for choosing between 2 adjacent quadrants not specified—could cause oscillation
- **Weight tuning**: DWA coefficients (α, β, γ, σ) not optimized; no sensitivity analysis provided
- **Kinematic constraints**: Robot motion model in DWA considers velocity/acceleration limits but coupling between linear/angular velocity ignored
- **Multi-robot scenarios**: No consideration of coordination or collision avoidance with other moving agents
- **3D environments**: Strictly 2D grid-based; vertical obstacles or multi-floor navigation not addressed

- Relevance to my Research: - *Direct A enhancement parallel*: Adaptive neighborhood expansion based on obstacle proximity is **directly applicable** to your Bidirectional A* obstacle penalties—can define penalty zones that trigger expansion strategy changes

- **Quadrant selection for urban context**: Coordinate-difference directional bias (simpler than angle calculation) can **accelerate forward/backward search convergence** in Bidirectional A* by biasing expansion toward meeting point
- **Redundant point elimination**: Line-of-sight pruning strategy provides **actionable post-processing method** to smooth your Bidirectional A* paths without replanning
- **Hybrid global-local framework**: A* + DWA architecture demonstrates **proven integration pattern**—your Bidirectional A* could similarly output waypoints for local reactive layer (relevant for real-time urban navigation with pedestrians/vehicles)
- **Obstacle-aware penalty calibration**: Their adaptive switching (4-neighbor near obstacles, 8-neighbor in open space) suggests your **penalty function should scale with obstacle density**—higher penalties in cluttered intersections, lower on open roads
- **Performance baseline**: 70% node reduction and <10s runtime on 20×20 grid sets **computational efficiency target** for your method in comparable complexity environments
- **Contrast with Bidirectional advantage**: Unidirectional A* required 12 expansions; your Bidirectional A* should theoretically achieve **$\sqrt{2}$ speedup** (expand from both ends)—but must handle obstacle penalties symmetrically in forward/backward search
- **Urban navigation mapping**: Quadrant method assumes direct line toward goal—works for open grids but **may need modification** for urban road networks with forced turns (one-way streets, barriers)

- Paper File Name : applsci-13-04290-v2.pdf

- Paper Name: nan

- Citation: Li, J., Kang, F., Chen, C., Tong, S., Jia, Y., Zhang, C., & Wang, Y. (2023). The Improved A* Algorithm for Quadrotor UAVs under Forest Obstacle Avoidance Path Planning. Applied Sciences, 13(7), 4290.

- Problem/Gap: - **xcessive traversed nodes**: Traditional A* searches too many nodes, reducing efficiency in forest environments
- **Redundant turning points**: Multiple unnecessary turns increase UAV energy consumption and flight instability
- **Large turning angles**: Sharp turns compromise flight continuity and safety in plantation forests
- **Path roughness**: Unsmooth trajectories unsuitable for continuous UAV operation
- **Local optima in pruning**: Existing redundant point removal strategies (connecting only to turning points) miss optimal shortcuts
- **Unified weight limitations**: Fixed g(n)/h(n) weights fail to balance convergence speed with path quality across search stages

- Algorithm/Method: - **Segmented evaluation function with dynamic weights**:
  - Stage 1 ($h(n) \geq L/3$): $f(n) = 2g(n) + (1 + d/L)h(n)$ — accelerates search with dynamic

heuristic boost
  - Stage 2 ($h(n) < L/3$): $f(n) = g(n) + (d/L)h(n)$ — refines accuracy near goal
  - Weight factor d/L decreases as UAV approaches target (d = current-to-goal distance, L = start-to-goal distance)
- **Steering cost heuristic**: $h(n) = (1 + k\times\beta)h_d$, where $\beta \in [0,\pi]$ is angle between current heading and goal direction
  - Penalizes large heading deviations to minimize turning frequency
  - β calculated via dot product: $\beta_i = \cos^{-1}(U_P U_g \cdot U_P U_{pi} / ||\text{vectors}||)$
- **Forward-backward redundant point removal**:
  - Connects goal node P_g directly to start P_s; if collision, iteratively connects to intermediate non-turning-point nodes
  - Bidirectional pruning (goal→middle, middle→start) avoids local optima from sequential turning-point-only connections
- **Quasi-uniform cubic B-spline smoothing**: Control point search radius $R \le 0.6r$ (r = grid size) prevents collision during curve fitting

- Heuristic/Obstacle Handling: - **Angle-aware cost**: Steering penalty $k\times\beta\times h_d$ guides robot toward goal direction while avoiding sharp turns
- **Dynamic heuristic weighting**: Factor $(1 + d/L)$ adaptively strengthens heuristic influence when far from goal, weakens when close
- **Collision detection during pruning**: Line-of-sight checks ensure shortcut paths avoid obstacles
- **B-spline constraint**: Limits smoothing radius to maintain safe clearance from obstacles after curve fitting
- **Segmented strategy balance**: Early search prioritizes speed (higher heuristic weight), final approach prioritizes accuracy (standard Dijkstra-like weighting)

- Key Results : **Performance across 4 grid environments (20×20 to 50×50, 40% obstacles):**

- **Traversed nodes**: 64.87% average reduction (e.g., 1166→243 in 50×50 grid)
- **Search time**: 49.64% average reduction (7.77s→4.55s in 50×50 grid)
- **Path length**: 12.52% average reduction
- **Total turning angle**: 54.53% average reduction (2925°→1525° in 50×50 grid)
- **Turning points**: Eliminated to zero after full optimization (48→0 in 50×50 grid)

Real plantation forest map (246×200 grid) comparisons:

- vs. Traditional A*: 33.53% faster search, 6.00% shorter path, 92.17% less turning angle
- vs. RRT: 95.34% faster, 30.69% shorter, 98.95% less turning
- vs. APF: 97.07% faster, 1.58% shorter, 94.05% less turning
- vs. Improved A* [ref 38]: 20.89% faster, 3.98% shorter, 90.90% less turning

- Limitations/Open Issues: - **Static environment assumption**: Algorithm not tested with moving obstacles or dynamic forest changes
- **Parameter sensitivity**: Optimal weights (a=2, b=1) derived empirically for 40% obstacle density—unclear if generalizable to other densities
- **Computational overhead**: B-spline smoothing and bidirectional pruning add processing time (though still net reduction overall)
- **3D simplification**: Uses 2D projection at fixed flight height; doesn't handle vertical obstacles (branches) or terrain elevation changes
- **Grid resolution dependency**: Performance tied to 1×1 grid size; unclear how algorithm scales with finer/coarser discretization
- **Collision radius unclear**: Paper doesn't specify UAV safety radius or how obstacle inflation is handled
- **Real-time applicability**: Longest test (50×50 grid) took 4.55s—may not meet <1s requirements for high-speed UAV flight
- **Kinematic constraints ignored**: No consideration of UAV acceleration limits, minimum turn radius, or velocity continuity

- Relevance to my Research: - **Segmented weighting paradigm**: Dynamic d/L factor directly applicable to *Bidirectional A meeting-point heuristics*—can adjust obstacle penalties as forward/backward searches converge (higher penalties early, lower near meeting zone)
- **Steering cost integration**: β-based angle penalty demonstrates **actionable method to penalize sharp turns** in urban navigation—your Bidirectional A* could add similar heading deviation cost to avoid jagged paths at intersections
- **Bidirectional pruning insight**: Forward-backward redundant point removal suggests **symmetric optimization for Bidirectional A**—both search directions should apply obstacle penalties consistently to avoid asymmetric meeting points
- **Stage-based search strategy**: Segmented evaluation function (aggressive heuristic initially, conservative near goal) maps to **urban navigation phases** (long straightaways vs. complex intersections)—obstacle penalties should scale with environmental complexity
- **Quantitative penalty calibration**: Optimal weight a=2, b=1 provides **empirical baseline** for tuning your obstacle-aware penalty coefficients
- **Real-world validation approach**: Plantation forest 3D LiDAR→Octomap→2D grid pipeline offers **replicable methodology** for testing your algorithm on real urban point cloud data (street scenes, pedestrian zones)
- **Performance benchmark**: 64.87% node reduction and 49.64% time savings set **efficiency targets** for your Bidirectional A* enhancements
- **Contrast with unidirectional**: Paper's unidirectional A* required 1166 nodes (50×50); Bidirectional A* should theoretically achieve **$\sqrt{2}$ speedup** (expand from both ends)—but obstacle penalties must be symmetric
- **Smoothness trade-off**: B-spline smoothing increased path length by ~1-2%—suggests **post-processing penalty adjustment** may be needed if your real-time constraints are tight
- **Urban navigation adaptation**: Forest-specific steering cost (avoid sharp turns for UAV

stability) parallels **urban vehicle constraints** (prefer wide turns at intersections, avoid sudden lane changes)—directly transferable concept

- Paper File Name : electronics-11-03660.pdf

- Paper Name: nan

- Citation: Liu, L., Wang, B., & Xu, H. (2022). Research on Path-Planning Algorithm Integrating Optimization A-Star Algorithm and Artificial Potential Field Method. Electronics, 11(22), 3660.

- Problem/Gap: - **Excessive computation**: Traditional A* returns to start when encountering obstacles, generating many useless nodes and redundant computation
- **Path rigidity**: Planned paths have too many turning points requiring frequent acceleration/deceleration
- **APF local minima**: Traditional artificial potential field method easily trapped in local optima, unreachable targets
- **Target unreachability**: When obstacles near target, repulsive force >> gravitational force prevents robot from reaching goal
- **Path discontinuity**: Direct APF smoothing creates discontinuous paths at global turning points
- **Memory overhead**: Breadth-first expansion increases useless memory usage and pathfinding time

- Algorithm/Method: - *Optimized A structure modification*:
  - When obstacle encountered: stays in place, sets obstacle node cost to ∞ (instead of returning to start)
  - Selects best child node as next parent for continued pathfinding
  - Eliminates backtracking to start point
- **Interruption point strategy for APF**:
  - Uses A* global path turning points as temporary start/endpoints for APF
  - Turning point judgment: $K_1 = (Xc - Xc\text{-}1)/(Yc - Yc\text{-}1)$, $K_2 = (Xc+1 - Xc)/(Yc+1 - Yc)$; if $K_1 \neq K_2$, current point is turning point
  - Manhattan distance determines when intermediate point passed
- **Adaptive iteration count**: $I = L \times \sqrt{[(Rx - TEx)^2 + (Ry - TEy)^2]}$, where L=10 (reference value per unit Euclidean distance)
- **Least squares path fitting**: Solves APF path discontinuity at turning points via polynomial fitting $\varphi(x) = a_0 + a_1x + ... + akx^k$
- **Hybrid workflow**: Global A* → Segment into turning points → Local APF smoothing between segments → Least squares continuous fitting

- Heuristic/Obstacle Handling: - **Cost function**: f(n) = g(n) + h(n), using Euclidean distance for both g(n) and h(n)
- **Obstacle node penalty**: Sets f(n) = ∞ for obstacles during search

- **APF gravitational field**: $U_{att}(x) = \frac{1}{2}K\rho^2(P_s, P_E)$; $F_{aat}(x) = -K\rho(P_s, P_E)$ (proportional to distance)
- **APF repulsive field**:
  - $U_{rep}(x) = \frac{1}{2}K_{rep}[1/\rho(P,P_{obs}) - 1/P_0]^2$ when $\rho \le P_0$; 0 otherwise
  - $F_{rep}(x) = K_{rep}[1/\rho - 1/P_0] \times 1/\rho^2$ (inversely proportional to obstacle distance)
- **Combined force**: $F_{sum}(x) = F_{aat}(x) + \Sigma F_{rep}(x)$ guides robot in real-time
- **Limited force range**: Repulsive force only applied within certain radius centered on robot (reduces computation)
- **Dynamic obstacle avoidance**: APF real-time map updates enable reactive avoidance while smoothing

- Key Results : **Simple environment (20×20 grid):**

- **vs Traditional A**: 60% reduction in path-planning time (0.603s→0.174s average)
- **vs Bidirectional A**: 65.2% time reduction, 32% fewer computation nodes (103→70)

Complex environment (40×40 grid):

- **vs Traditional A**: 70% time reduction (2.368s→0.692s), same path quality

Dynamic obstacles (20 random obstacles):

- Successfully avoided all obstacles at points (8,9) and (12,8); reached target safely

Algorithm comparisons (fusion vs others):

- **vs Ant Colony**: 65.2% time reduction (2.073s→0.722s average)
- **vs RRT**: 83.64% time reduction (4.003s→0.655s average)
- Fusion algorithm maintains path smoothness with no obvious turning points

Iteration parameter L sensitivity:

- L=1: 30% faster than L=10 but fails to reach interim endpoints
- L=100: 288% slower than L=10 with no quality improvement
- **Optimal L=10**: Balances time and completion rate

- Limitations/Open Issues: - **Static global assumption**: A* global plan doesn't update; relies entirely on APF for dynamic obstacles
- **Parameter tuning burden**: Requires manual selection of K (gravitational coefficient), K_{rep} (repulsive coefficient), P_0 (critical distance), L (iteration reference)
- **Path fitting overhead**: Least squares adds computational step; paper doesn't report fitting time separately
- **Turning point dependency**: Fusion performance degrades if global A* generates suboptimal turning points

- **No kinematic constraints**: Ignores robot acceleration limits, turning radius, velocity continuity
- **Narrow passage limitation**: APF repulsive force may prevent passage through tight corridors even with A* guidance
- **Scalability unclear**: All tests on small grids (≤40×40); computational complexity for large urban maps not addressed
- **Real-time guarantees missing**: No worst-case time bounds; 0.6-0.7s average may not meet high-speed navigation requirements
- **APF oscillation near goals**: Traditional APF goal oscillation issue not fully addressed—only mitigated by using turning points as temporary goals

- Relevance to my Research: - **Hybrid global-local paradigm**: A* (global) + APF (local) architecture **directly mirrors** Bidirectional A* + local reactive layer—your enhanced A* could output waypoints for APF-style local adjustments
- **Turning point as decision nodes**: Using global path turning points as "interruption points" suggests *Bidirectional A meeting point should trigger penalty recalibration*—switch from exploration mode to refinement mode
- **Obstacle cost calibration**: APF's Krep coefficient and P_0 critical distance provide **concrete penalty structure**—your obstacle-aware penalties could mimic repulsive field decay (higher near obstacles, zero beyond threshold)
- **Stay-in-place strategy**: Optimized A* setting obstacle cost to ∞ (vs backtracking) demonstrates **computational efficiency gain**—your Bidirectional A* should similarly avoid redundant re-expansion
- **Adaptive iteration concept**: I = L × distance formula shows **dynamic parameter adjustment based on search progress**—your penalty weights could scale with remaining distance to meeting point
- **Path discontinuity problem**: Least squares fitting requirement highlights **post-processing need**—your Bidirectional A* may need similar smoothing between forward/backward segments at meeting point
- **Performance baseline**: 60-70% time reduction over traditional A* sets **efficiency target**; Bidirectional A* should theoretically match/exceed this
- **Dynamic obstacle handling**: APF real-time updates suggest **local penalty adjustment mechanism** for your Bidirectional A* when new obstacles appear mid-search
- **Contrast with Bidirectional advantage**: Paper's unidirectional A* required 0.174s (20×20); *Bidirectional A should achieve sub-0.1s*on similar grids if obstacle penalties don't create asymmetric meeting issues
- **Urban navigation adaptation**: APF's repulsive force (push away from obstacles) parallels **urban safety margins**—sidewalks, building clearances, pedestrian zones should have similar gradient penalties
- **Computational trade-off insight**: L=100 (288% slower) vs L=10 demonstrates **diminishing returns from over-optimization**—your obstacle penalty tuning should stop at "good enough" threshold

- Paper File Name : noroozi2022

- Paper Name: nan

- Citation: Noroozi, M., Mohammadi, H., Efatinasab, E., Lashgari, A., Eslami, M., & Khan, B. (2022). Golden Search Optimization Algorithm. IEEE Access, 10, 37515-37532.

- Problem/Gap: ##

- **Classical algorithm limitations**: Gradient-based methods demand exponential time or fail with discontinuities, incomplete information, dynamicity, uncertainties
- **No Free Lunch theorem**: No single metaheuristic provides superior performance for all optimization problems
- **Premature convergence**: PSO performs poorly on high-dimensional, complex, hybrid problems
- **Exploration-exploitation balance**: Need for fine balance between global exploration and local exploitation phases
- **Parameter complexity**: Many algorithms require extensive parameter tuning

- Algorithm/Method: - **Sine/cosine oscillation**: Allows re-positioning around solutions; guarantees exploitation between two solutions
- **Extended range**: Increasing sine/cosine range enables position updates outside space between current and target (exploration)
- **Adaptive step control**: Transfer operator T exponentially decays (100→0), strengthening exploitation as iterations progress
- **Random worst replacement**: Injecting random solution instead of worst object prevents stagnation
- **Personal vs global guidance**: Balances individual experience (Obest_i) with swarm knowledge (Ogbest_i)
- **No explicit obstacle handling**: Algorithm designed for continuous function optimization, not path planning with physical obstacles

- Heuristic/Obstacle Handling: - Sine/cosine oscillation**: Allows re-positioning around solutions; guarantees exploitation between two solutions
- *Extended range**: Increasing sine/cosine range enables position updates outside space between current and target (exploration)
- **Adaptive step control**: Transfer operator T exponentially decays (100→0), strengthening exploitation as iterations progress
- **Random worst replacement**: Injecting random solution instead of worst object prevents stagnation
- **Personal vs global guidance**: Balances individual experience (Obest_i) with swarm knowledge (Ogbest_i)
- **No explicit obstacle handling**: Algorithm designed for continuous function optimization, not path planning with physical obstacles

- Key Results : **Unimodal functions (F1-F7, exploitation test):**

- **Best performance**: Superior on 6/7 functions; reached global optima on F1-F4
- vs GSA/SCA/TSA/GWO: Better mean values and standard deviations

Multimodal functions (F8-F23, exploration test):

- **Best performance**: Superior on 11/16 functions (F8-F11, F14-F16, F20-F23)
- **Comparable**: F17-F19 comparable to competitors
- **Slightly inferior**: F12-F13 mean values better than GSA, much better than SCA/TSA/GWO
- **Stability**: Lower standard deviations indicate more consistent convergence

High-dimensional (100D) scalability:

- **Outperforms all competitors** on all 13 scalable functions (F1-F13) at 100 dimensions
- Demonstrates efficiency as problem complexity increases

Statistical significance (Wilcoxon rank sum test, α=0.05):

- **vs GSA**: Superior in 17/23 cases, inferior in 5/23, equivalent in 1/23
- **vs SCA/TSA/GWO**: Majority of test suite shows statistically significant superiority

Time complexity: O(t_max × (N×D + N×F(X))) — comparable to standard metaheuristics

- Limitations/Open Issues: - **No actual inspiration**: Uses simple mathematical functions (not bio-inspired)—may limit intuitive understanding or improvement directions
- **Parameter sensitivity**: Requires tuning of N (population size), MaxIter (iterations), C_1, C_2 coefficients
- **Optimal population size**: N=30 determined empirically; lower causes premature convergence, higher increases computation time
- **Fixed step limitation**: St_imax = 0.1 × bounds is constant; no adaptive mechanism for narrow corridors or local refinement
- **No constraint handling**: Designed for unconstrained continuous optimization; doesn't address constrained, discrete, or combinatorial problems
- **Random replacement strategy**: Replacing worst with random solution may disrupt convergence in late iterations
- **Sine/cosine periodicity**: May cause oscillations near optima without proper damping
- **No diversity maintenance**: Beyond worst replacement, lacks explicit diversity preservation mechanisms
- **Benchmark-only validation**: Tested on mathematical functions; no real-world engineering applications demonstrated
- **Transfer operator tuning**: T = 100 × exp(-20t/t_max) coefficients (100, -20) not justified; may not generalize to all problem types

- Relevance to my Research: - **Not directly applicable to path planning**: GSO designed for continuous function optimization, not discrete grid-based navigation with obstacles
- **Step size concept for Bidirectional A**: Transfer operator T (exponential decay 100→0) suggests **dynamic penalty weight reduction** as forward/backward searches approach meeting point
- **Exploration-exploitation balance**: Early strong exploration (T=100) → late exploitation (T→0) parallels *initial aggressive A expansion → refined local search*near meeting zone
- **Sine/cosine oscillation insight**: Oscillatory behavior for re-positioning could inspire **local perturbation strategy** around Bidirectional A* meeting point to escape suboptimal junctions
- **Personal vs global best**: Obest_i (personal) + Ogbest_i (global) balance maps to **forward search history + backward search history** in Bidirectional A*—penalties should consider both directions' findings
- **Random worst replacement**: Replacing worst solution with random one suggests **periodic path re-evaluation** in Bidirectional A*—discard dead-end branches, inject exploration
- **Adaptive step limitation**: St_imax = 0.1 × bounds provides **concrete penalty bound formula**—urban obstacle penalties could scale as 10% of max grid distance
- **Statistical validation approach**: Wilcoxon rank sum test methodology applicable to comparing your Bidirectional A* vs traditional A*/RRT/APF on urban maps
- **Convergence curve analysis**: Fig. 6 comparison style useful for *visualizing Bidirectional A node expansion vs unidirectional*across iteration count
- **High-dimensional scalability**: 100D testing demonstrates **need to validate your algorithm** on large urban grids (>100×100) beyond small test cases
- **Contrast—continuous vs discrete**: GSO's continuous step size (St_i) fundamentally different from **discrete grid jumps** in A*—cannot directly port update equations
- **Contrast—no obstacle model**: GSO lacks physical constraints; your **obstacle-aware penalties must explicitly model** collision, clearance, safety margins
- **Time complexity benchmark**: O(t_max × N×D) sets **computational baseline**—your Bidirectional A* should aim for O(log N) speedup via bidirectional pruning

---